



LMCC

LMCC线上培训系列讲座

大模型架构与LMCC第一轮真题选讲

西安铁一中学 徐保毅

QQ: 465039728

AI 再一次突破人类认知边界

百年黎曼猜想被Claude破了新纪录！是个未公开新模型



一水

2026-08-11 13:12:20

来源：量子位



Anthropic

@AnthropicAI

We asked an unreleased research version of Claude to take a stab at the Riemann hypothesis.

It didn't solve it, but it did make strides on a related problem: it increased the lower bound for the fraction of zeros of the Riemann zeta function that satisfy the hypothesis from 41.6% to 67.2%.

我们让 Claude 的一个尚未公开发布的研究版本尝试解决黎曼假设问题。

虽然这并没有完全解决问题，但至少在相关问题上取得了一些进展：它将满足假设的黎曼 ζ 函数中零位数的最低阈值从 41.6%提高到了 67.2%。

AI证伪雅可比猜想：张益唐的遗憾与数学新范式

学习 LMCC 核心 - 培养工程思维 （不要为了考试而考试）

- NOI 学习基础算法，在限定的时间和空间里面解决问题。（科学性/学术性-理想主义者）

✓工程思维：像工程师一样思考解决问题

工程思维是一种在现实边界和资源限制下，追求可行性、系统性与商业闭环的务实解决问题能力，其核心在于用有限步骤和模块化手段达成可叠加的成功。

- 1. 在边界内做事：**承认理论和现实的极限，不去做徒劳的突破，在约束条件下寻求最优解。
- 2. 追求闭环与落地：**点子和理论只是起点，工程思维强调从0到1的完整全过程，最终要在市场上或实际中产生价值。
- 3. 化繁为简的解构：**把巨大而复杂的系统拆解成可理解、可处理的模块，再重新组合。

NOI 选手 应该注意培养的 “工程思维”



面对复杂问题，不要幻想一次性成功，寻找到系统 “迭代升级” 的方式



实际问题拆解，化繁为简，在 “理想-现实” 间寻找到可落地的 “方案”



查阅学习相关技术文档、参考手册、论文



敏锐的判断力 （善用 AI 工具，并判断） （方法论：交叉验证）

考纲重点： 模型架构

三. 模型架构

● 【1】 注意力机制

- 知识点：查询 (query) - 键 (key) - 值 (value), softmax函数
- 扩展知识点：自注意力机制、交叉注意力机制
- 考核方式：概念、计算

● 【1】 主流架构

- 知识点：编码器-解码器架构、因果解码器架构、前缀解码器架构
- 扩展知识点：架构选择与任务类型的关系（如生成任务 vs. 理解任务）
- 考核方式：概念、架构对比分析

● 【1】 Transformer模型的基本结构组成

- 知识点：输入编码、位置编码、多头自注意力机制、前馈网络层、编码器、解码器
- 扩展知识点：位置编码的变体（如相对位置编码、旋转位置编码）
- 考核方式：概念、公式、代码实现

● 【2】 Transformer模型的详细配置

- 知识点：归一化方法 (LayerNorm、RMSNorm)、归一化模块位置、激活函数 (GELU、Swish)、注意力机制
- 扩展知识点：不同配置对模型性能的影响、大模型可解释性分析方法（探针技术、注意力可视化、特征重要性分析）
- 考核方式：概念、公式、实现与分析

● 【3】 混合专家模型 (MoE)

- 知识点：稀疏激活、路由机制、负载均衡
- 扩展知识点：MoE的显存优化与通信开销
- 考核方式：概念、公式、实现

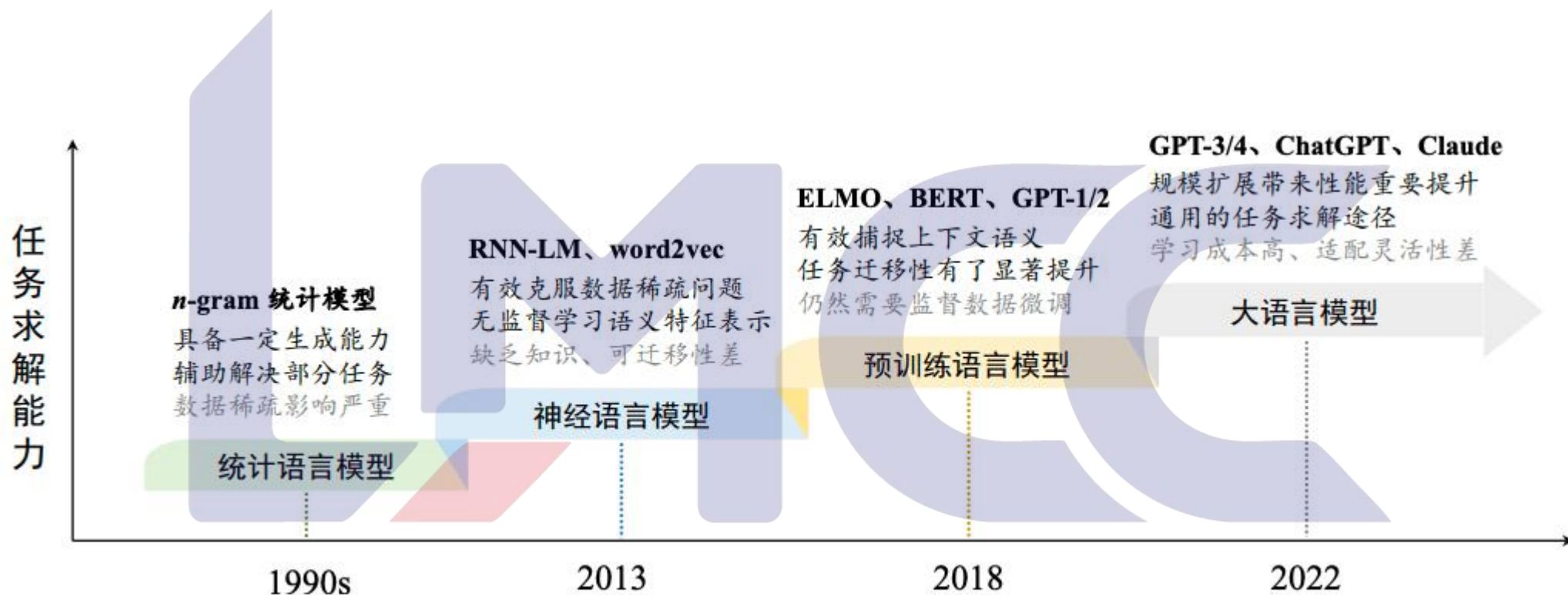
● 【3】 长上下文模型

- 知识点：位置编码拓展（如ALiBi）、长上下文窗口拓展、长上下文数据构建
- 扩展知识点：长上下文模型的显存与计算效率优化、长上下文模型的训练方法
- 考核方式：概念、方法、实现

● 【3】 高效注意力架构

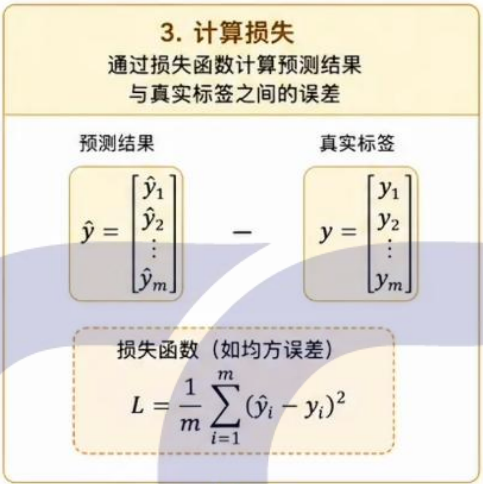
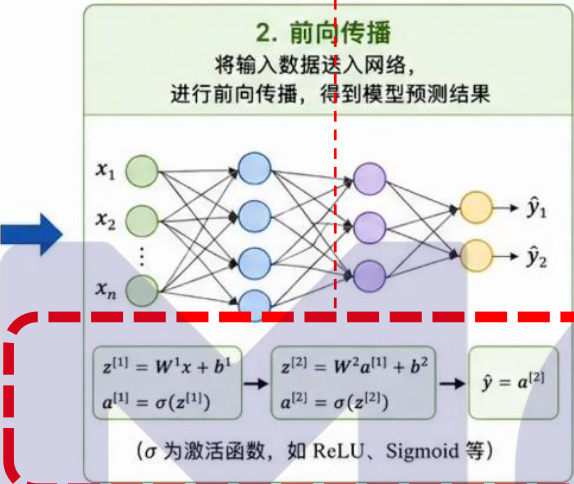
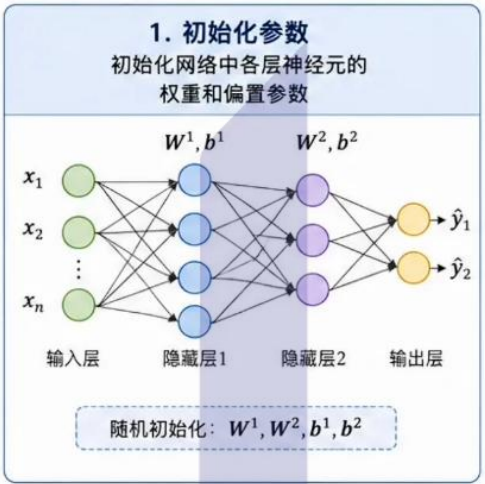
- 知识点：稀疏注意力、线性注意力、键值缓存优化、多查询注意力 (MQA)、分组查询注意力 (GQA)、多头潜在注意力 (MLA)
- 扩展知识点：高效注意力机制的理论基础（如低秩近似）
- 考核方式：概念、公式、实现

语言模型的发展



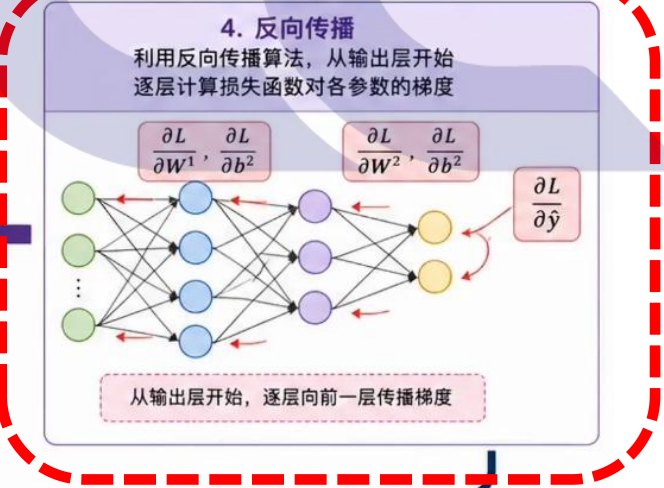
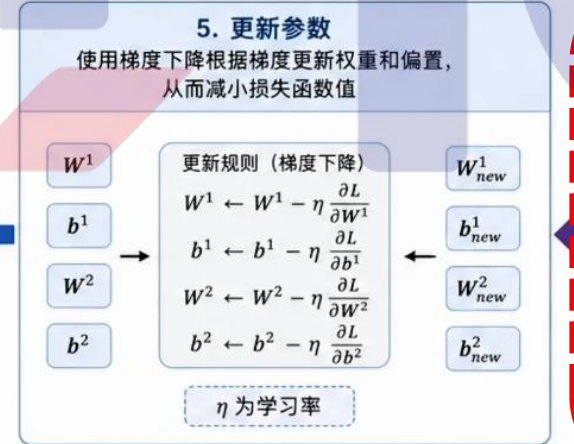
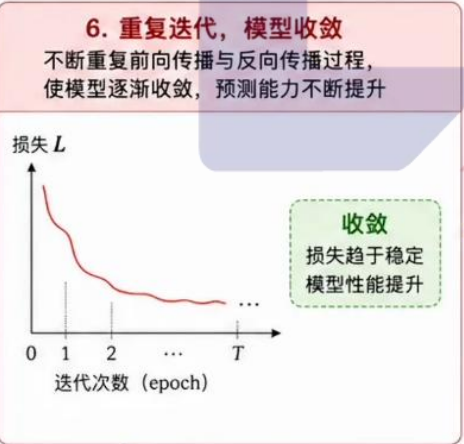
神经网络学习过程

激活函数



循环神经网络(RNN)
卷积神经网络(CNN)

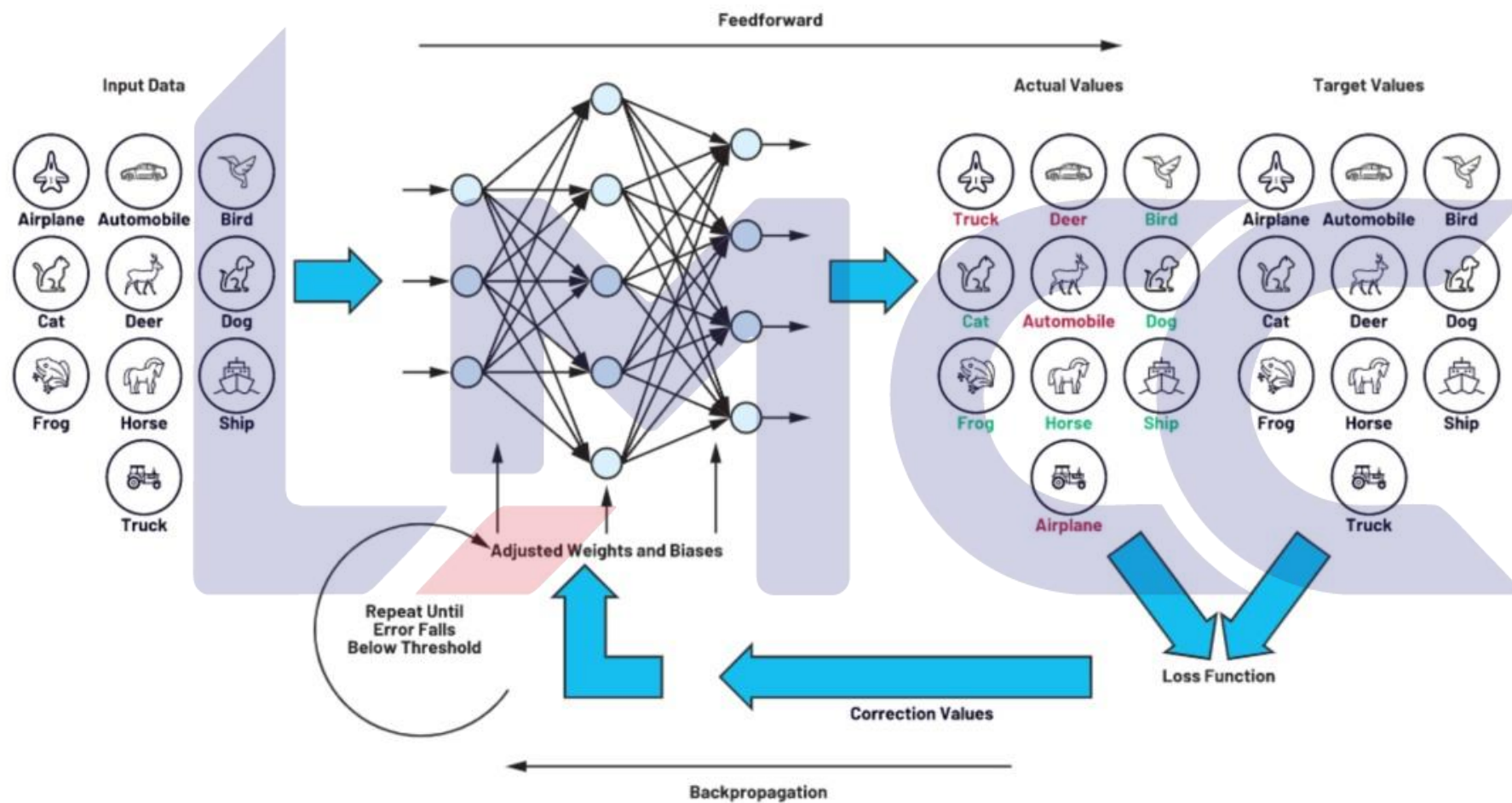
Transformer



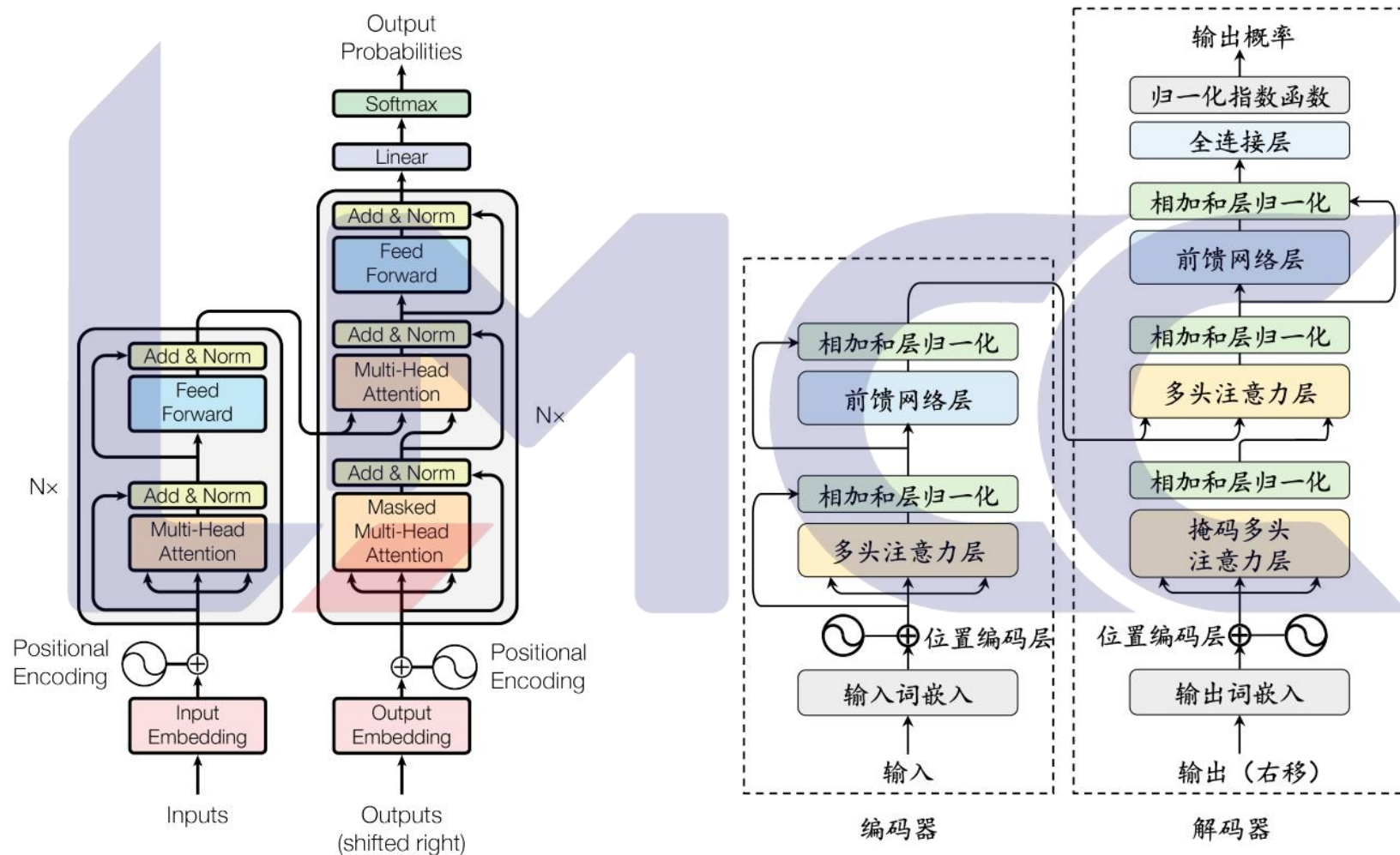
方向传播算法

重复迭代, 直到满足停止条件 (如损失收敛、达到最大迭代次数等)

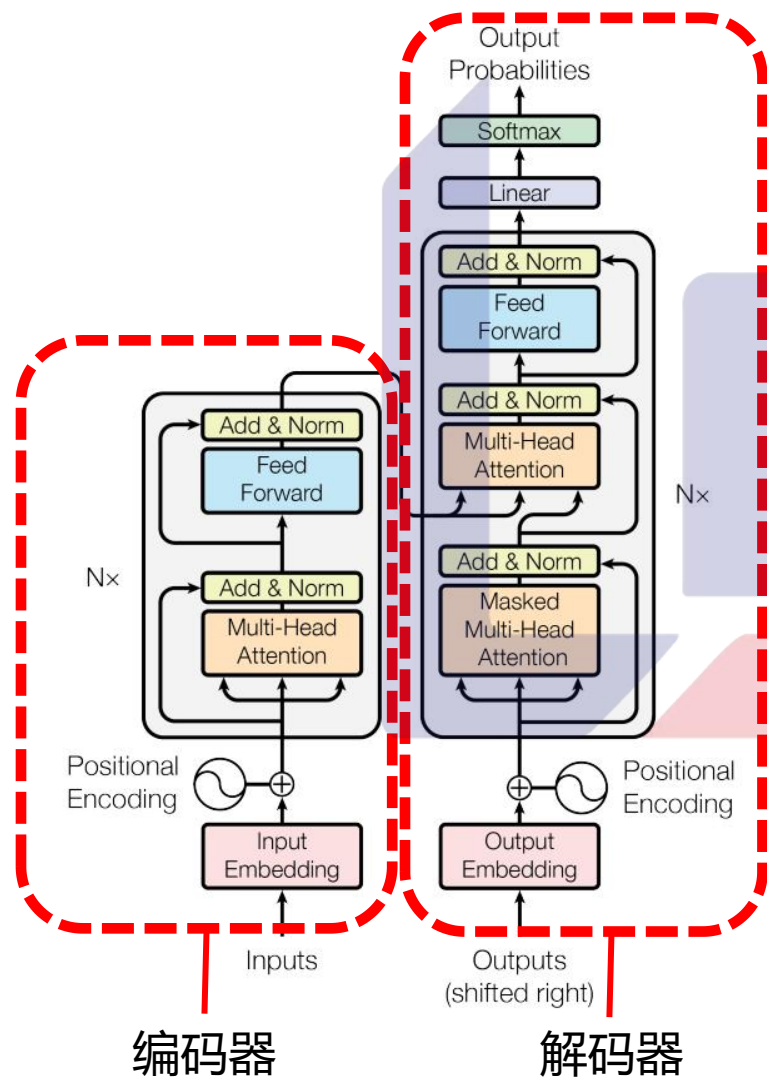
多层神经网络 - 机器学习



Transformer 模型架构



Transformer 架构：编码器和解码器



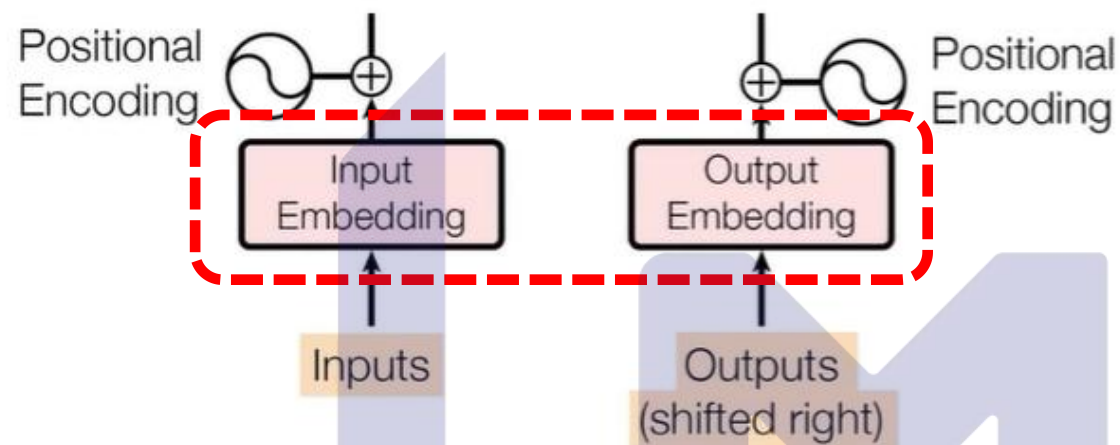
编码器 (Encoder)

负责理解和特征提取，“读懂”输入信息（文本/图片/视频）。由 N 层“编码器层”堆叠而成，每层包括两个核心子模块：多头注意力层、前馈网络层

解码器 (Decoder)

根据解码器读懂的语义，生成连贯、符合逻辑的输出信息。由 N 层“编码器层”堆叠而成，每层：掩码多头注意力层、多头注意力层（也叫作编码器-解码器注意力层，或者交叉多头注意力层）前馈网络层

1. 输入词嵌入 Embedding



文本序列: "Are you OK ?"



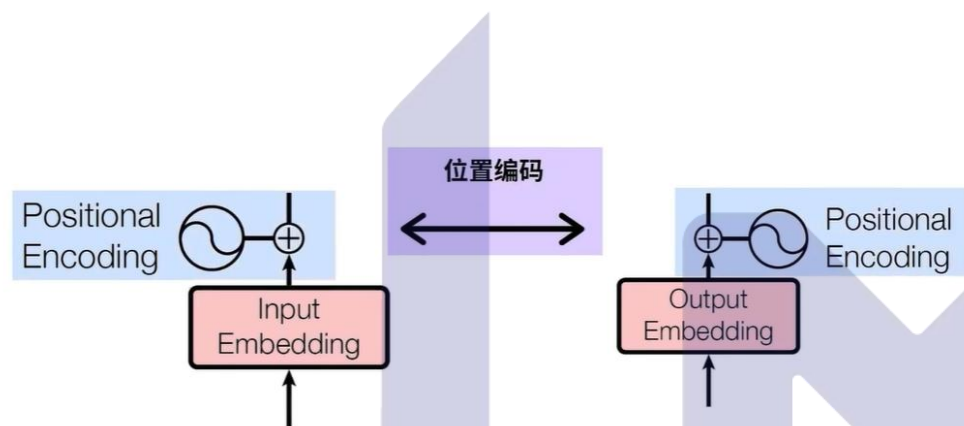
单词数字索引: [123, 456, 789, 10]

输入词嵌入是整个网络的第一步，其核心任务是将离散的 Token（如单词、子词或字符）映射到连续的高维稠密向量空间中，使得语义相近的词（如“猫”和“狗”）在空间距离上也更接近。

利用一个可学习的权重矩阵 $W_{emb} \in \mathbb{R}^{V \times d_{model}}$ （其中 V 为词表大小， d_{model} 为隐藏层维度），通过查表（Look-up）操作将每个 Token 的 ID 转换为对应维度的向量。

$$E = XW_{emb} \quad (E \in \mathbb{R}^{L \times d_{model}})$$

2. 位置编码 Positional Encoding



位置编码的目的是将**位置信息**附加到原始信息上。一个位置编码也是一个向量，其维度与词向量维度一样。将位置编码（Positional Embedding）和词本身的编码（Token Embedding）相加，即得到位置相关的编码向量。

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

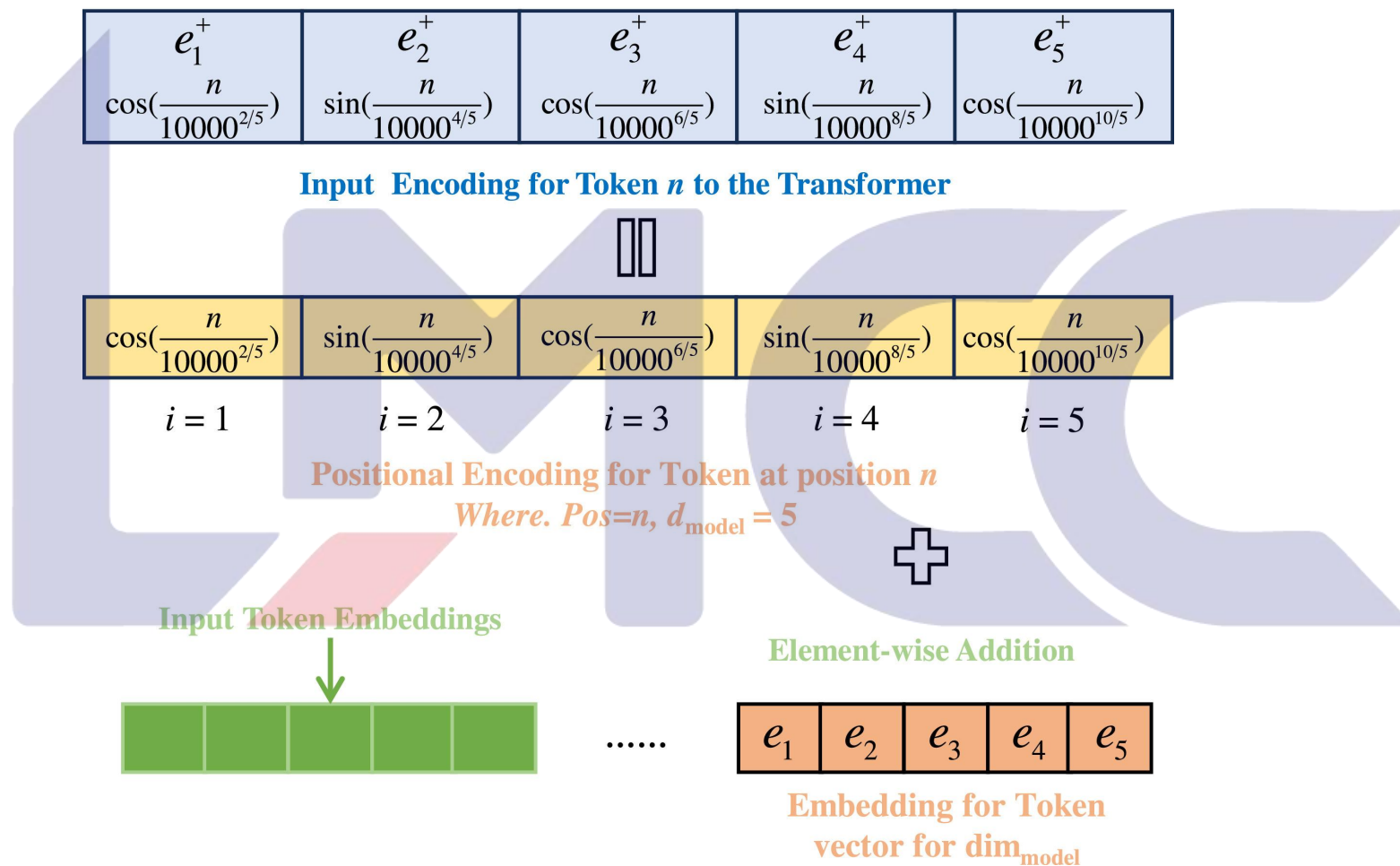
其中：

- pos 表示词在句子中的绝对位置（第几个词）。
- i 表示词向量的维度索引。
- d_{model} 是模型的总维度。

将位置编码矩阵 PE 与输入嵌入矩阵 E 直接按元素相加，作为后续编码器的输入。

$$X = E + PE$$

位置编码



论文中位置编码（PE）的改进

- 虽然原始的 PE 简洁有效，但也存在一些缺点。
 - 固定函数，缺乏灵活度。不能根据具体任务调整，表达能力弱于学习式编码。
 - 计算的是绝对位置，没有考虑到 词 之间的相对距离。



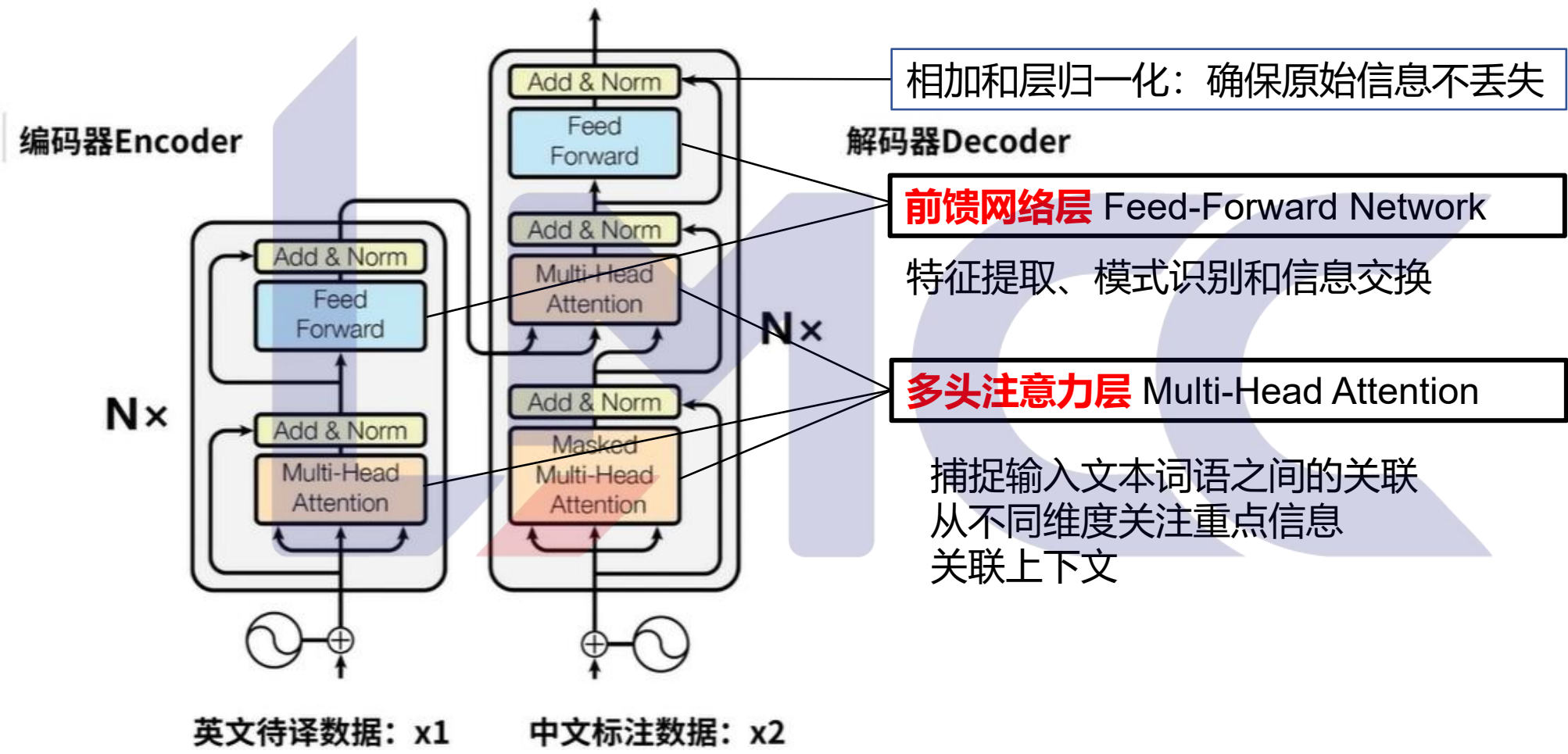
Positional_Encoding_Comparison.xls

方法	特点	应用模型
Learned PE	可学习的绝对位置编码	BERT、GPT-2、GPT-3
Relative PE	编码 token 间的相对位置	Transformer-XL, T5
RoPE (Rotary Position Embedding)	用绝对位置的旋转，实现相对位置的计算	LLaMA 系列、Grok、GLM、Qwen、Mistral (主流方法)
ALiBi (Attention with Linear Biases)	线性衰减偏置，在计算出的 Attention Score 矩阵上直接加上一个与距离呈线性衰减的惩罚偏置。	BLOOM、Falcon、MPT

位置编码总结与选型建议

- 1. 早期模型（BERT/GPT-2）：多采用 Learned PE，结构简单，但无法直接延伸上下文窗口。
- 2. 现代主流大语言模型（LLaMA/Qwen/GLM）：**绝大多数采用 RoPE，是当前兼顾效果、效率与长文本扩展能力的最佳平衡方案。**
- 3. 追求极佳外推性能场景（BLOOM/Falcon）：可考虑 ALiBi，推理时扩展上下文成本低，但对长距离精准记忆要求极高的任务需谨慎评估。

多头注意力和前馈网络层

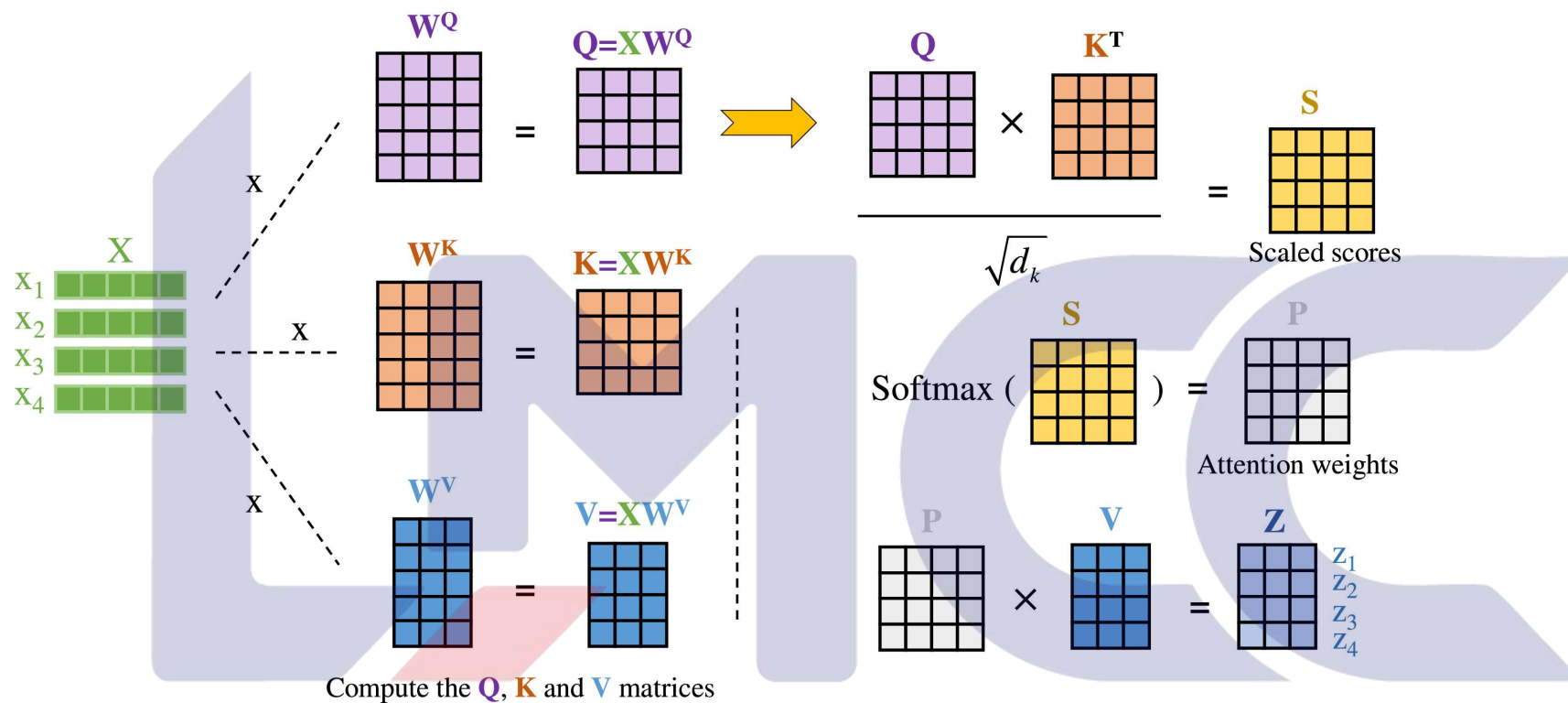


3. (重点) 自注意力机制 (Self-Attention)

- 2017 年 Google 在《Attention is All You Need》：**直接使用注意力机制编码整个文本**
- 循环神经网络(RNN)要逐步递归才能获得全局信息
- 卷积神经网络(CNN)只能获取局部信息，需要通过堆叠多层来增大感受视野
- **注意力机制一步到位获取了全局信息**
- 自注意力机制做的事情，和我们的大脑完全一样：它会让句子中的每个词语，都去“找”句子中与自己最相关的“朋友”，计算它们之间的关联程度，然后根据关联程度，分配不同的注意力权重-关联越紧密，注意力权重越高，就越能重点关注。
- 传统模型当句子很长时，模型会“忘记”前面提到的内容，无法准确理解后面词语的含义。
- 前沿的大模型研究工作很大一部分都是围绕着QKV矩阵
- **QKV权重占比着大语言模型50%以上的权重比例**，在推理过程中，QKV存储量还会随着上下文长度的增长而平方增长，计算量也平方增加。

注意力权重计算过程

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



可训练矩阵 W^q, W^k, W^v

Softmax: 多分类问题中的激活函数

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{c=1}^C e^{z_c}}$$

理解 QKV

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- 1. QK^T : 计算 Query 与所有 Key 的点积，衡量词与词之间的相关性分数。
- 2. $\sqrt{d_k}$: 缩放因子，防止点积结果过大导致梯度消失。
- 3. softmax: 将分数归一化为权重分布，表示当前词对其他词的关注程度。
- 4. 加权求和: 用归一化后的权重对 V 进行加权，得到当前位置的最终表示。

向量名称	通俗含义	数学含义
Query(查询)	Q代表当前位置希望获得的信息 找什么样的朋友？	用于与所有 K 向量乘积，计算相似度
Key(键)	K代表序列中各位置能提供的信息 人在不同维度下的特性 与Q 乘法，计算“亲密关系”	计算 Query 与所有 Key 的点积，衡量 关联关系。
Value(值)	V代表当查询和键匹配时，应该从各位置 获取的实际内容 整合朋友的其他信息	关联度计算完以后，用于加权求和，输出 最终结果

多头自注意力机制

1. 多层**并行计算**多个注意力头来捕捉**不同子空间的关联信息**。
2. **多个维度运算，信息互补，实测效果好。**

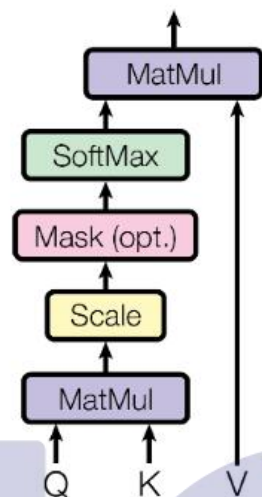
它将 Q, K, V 分成 h 个头，独立并行地进行注意力计算，最后将结果拼接并通过一个输出矩阵 W^O 进行融合。

$$MultiHead(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

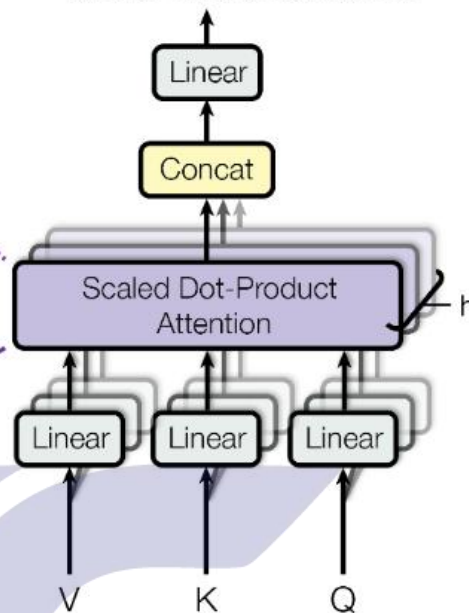
其中, $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$ 。

- W_i^Q, W_i^K, W_i^V 是第 i 个头的投影矩阵。
- W^O 是最后对拼接结果进行整合的线性输出矩阵。

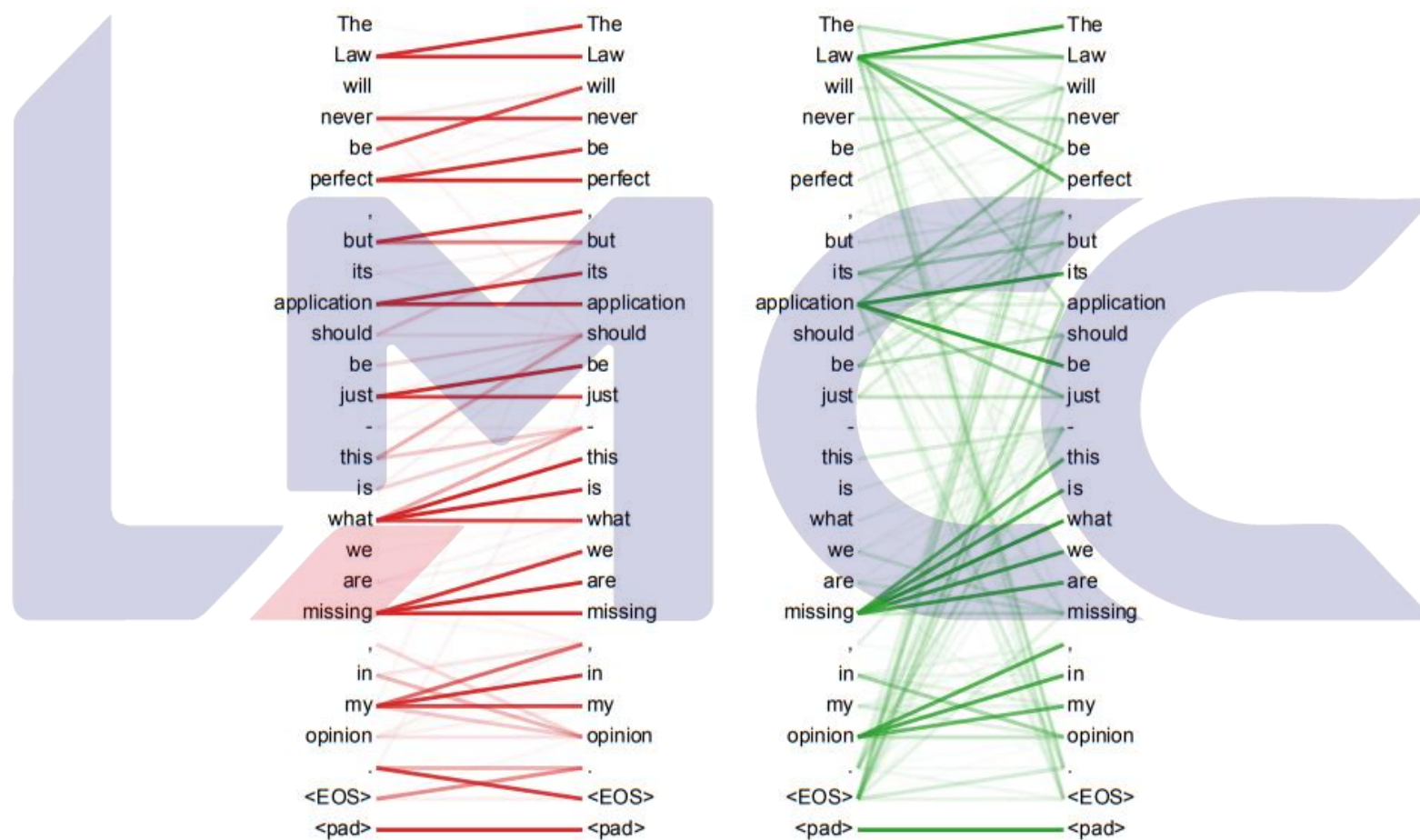
Scaled Dot-Product Attention



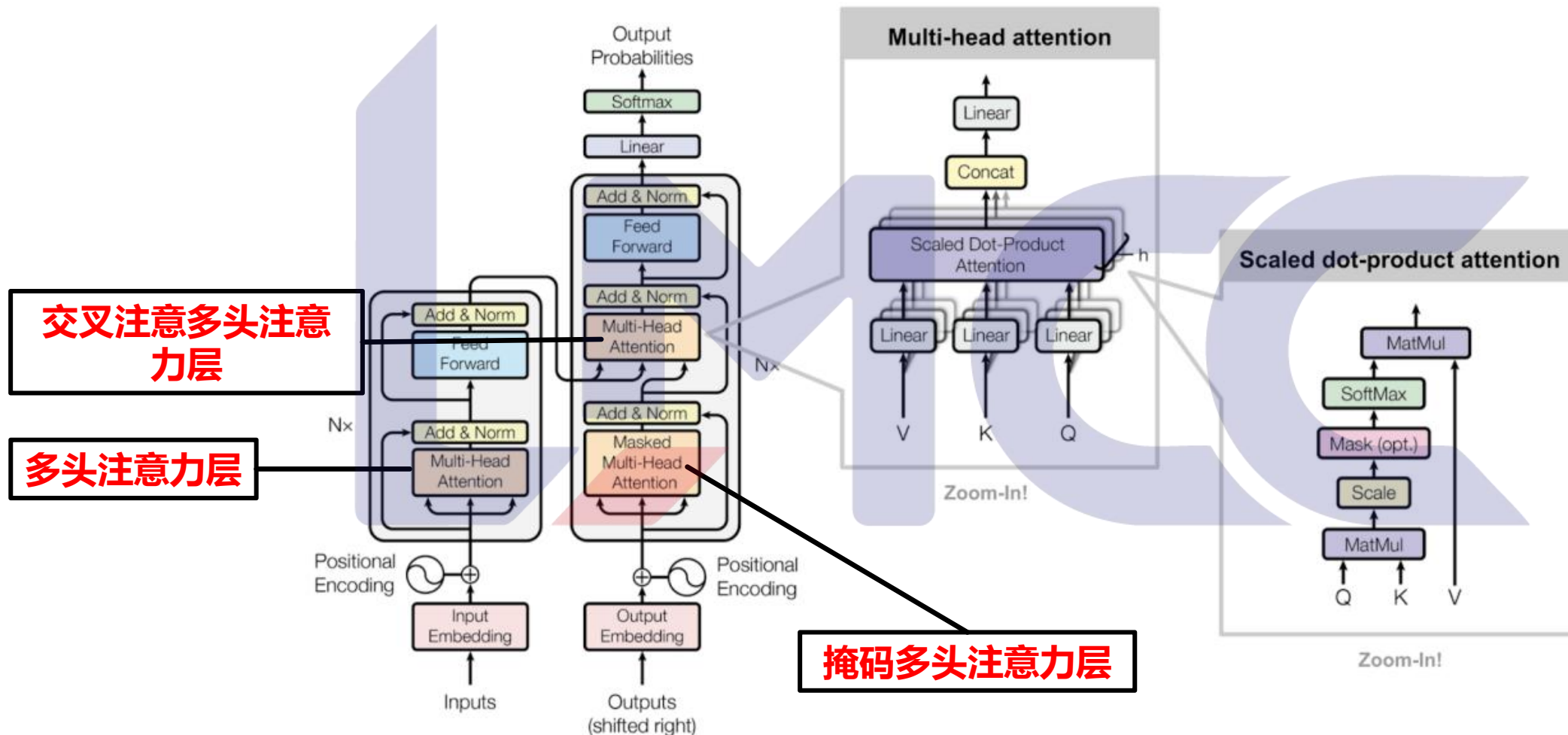
Multi-Head Attention



经过多头注意力机制后 token 之间的连接关系



多头注意力、掩码多头注意力、交叉注意力多头



为什么注意力机制如此重要？

- ✓优势1： 全局视野，轻松处理长文本
- ✓优势2： 并行计算，大幅度提升效率
- ✓优势3： “多头注意力”就是让模型同时从多个“角度”关注文本，捕捉更多样的关联信息。

4. 前馈网络层 FFN

- 前馈网络层由两个线性变换和一个非线性激活函数组成：

$$FFN(\mathbf{X}) = \sigma(\mathbf{X}\mathbf{W}^U + \mathbf{b}_1)\mathbf{W}^D + \mathbf{b}_2$$

激活函数

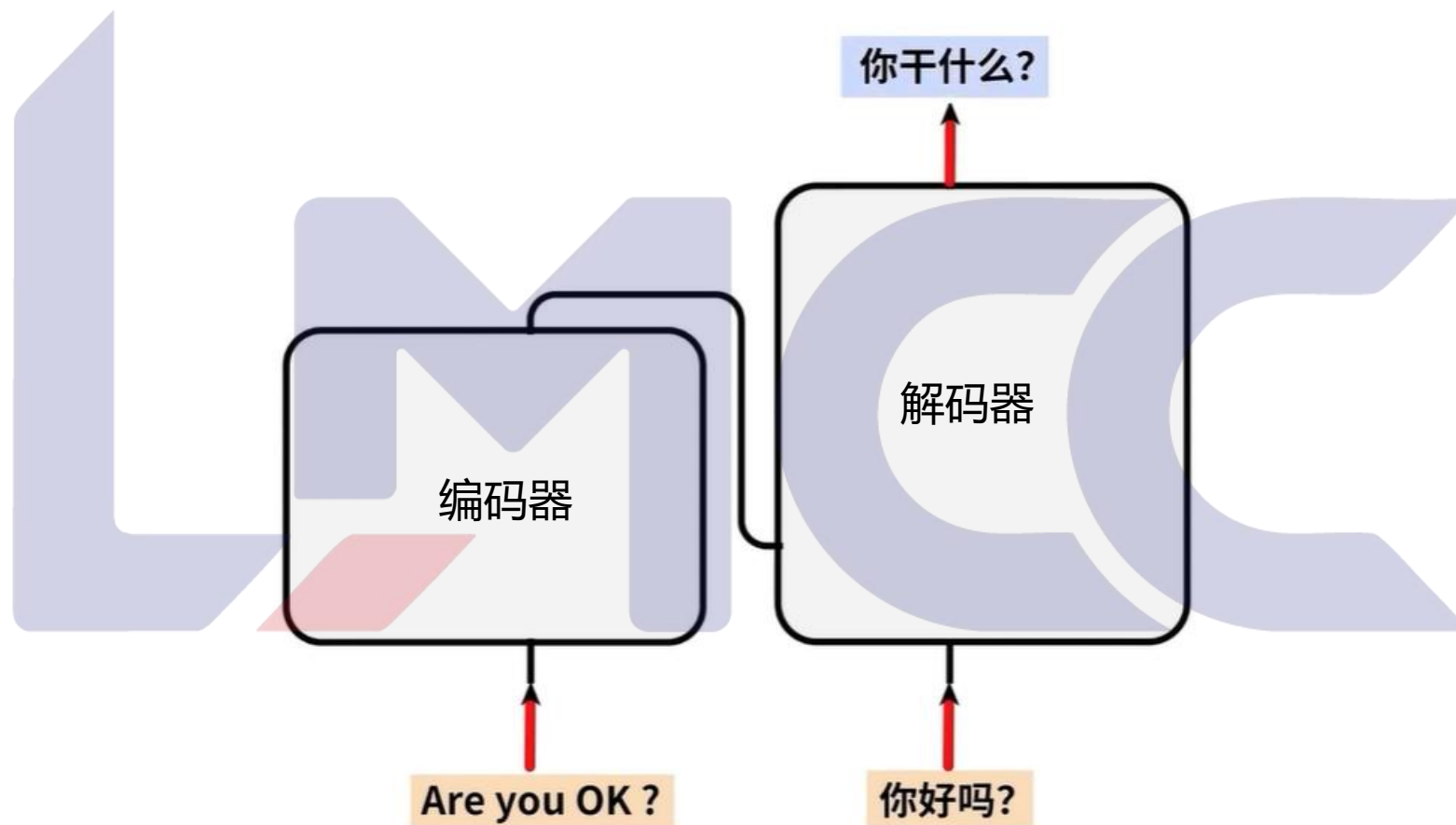
权重矩阵

偏置函数

1. $\mathbf{X}$ 通过权重矩阵 $\mathbf{W}^U$ 和偏置 $\mathbf{b}_1$ 将输入映射到更高维的空间，即对输入内容放大。
2. 通过激活函数 σ 将输入进行非线性映射变换，即转换为曲线，使能够捕捉更多的特征。
3. 通过权重 $\mathbf{W}^D$ 和偏置函数 $\mathbf{b}_2$ ，将输出映射回原维度，即将放大的数据进行缩小。

作用：特征提取、模式识别和信息变换

特征是如何抽取的？ 如何理解？



一个例子：机器翻译

Transformer 的训练模式：根据梯度，反向修改参数

中文标注

$Y = (\text{你, 好, 吗, ?})$

模型预测

$\hat{Y} = (\text{你, 干, 什, 么, ?})$

损失计算

交叉熵误差

$$\text{Loss} = \text{CrossEntropy}(Y, \hat{Y}) = - \sum_{i=1}^n y_i \log(\hat{y}_i)$$

完成这些计算后, 就完成了一轮迭代

$$\nabla_{\theta} \text{Loss} = \left(\frac{\partial \text{Loss}}{\partial \theta_1}, \frac{\partial \text{Loss}}{\partial \theta_2}, \dots, \frac{\partial \text{Loss}}{\partial \theta_n} \right)$$

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \text{loss}$$

反向传播算法

计算loss关于Transformer中参数 θ 的梯度

使用梯度下降算法, 更新这些参数 θ

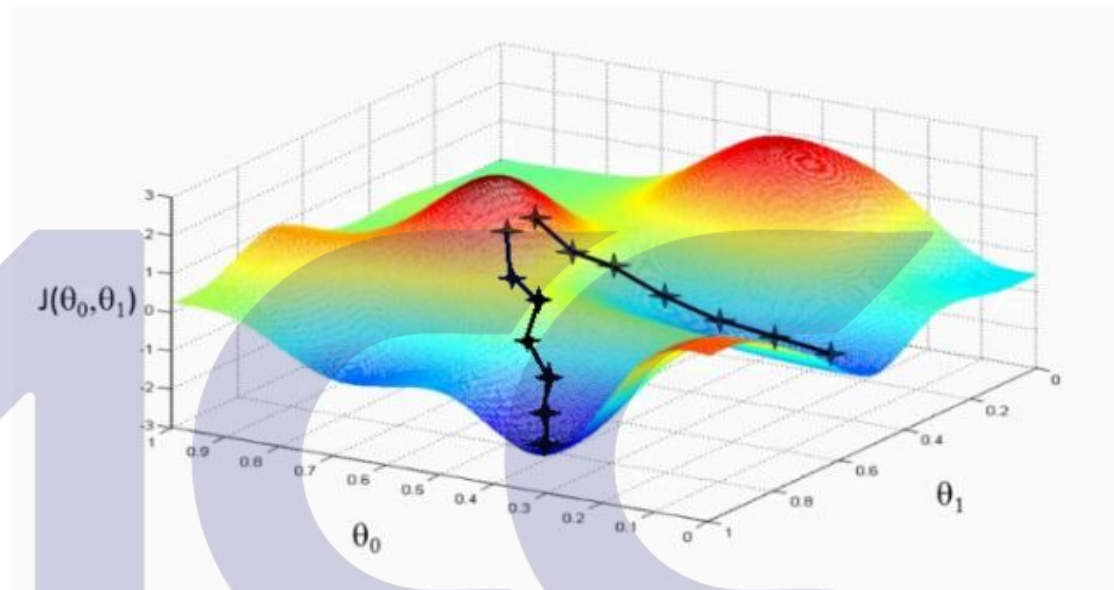
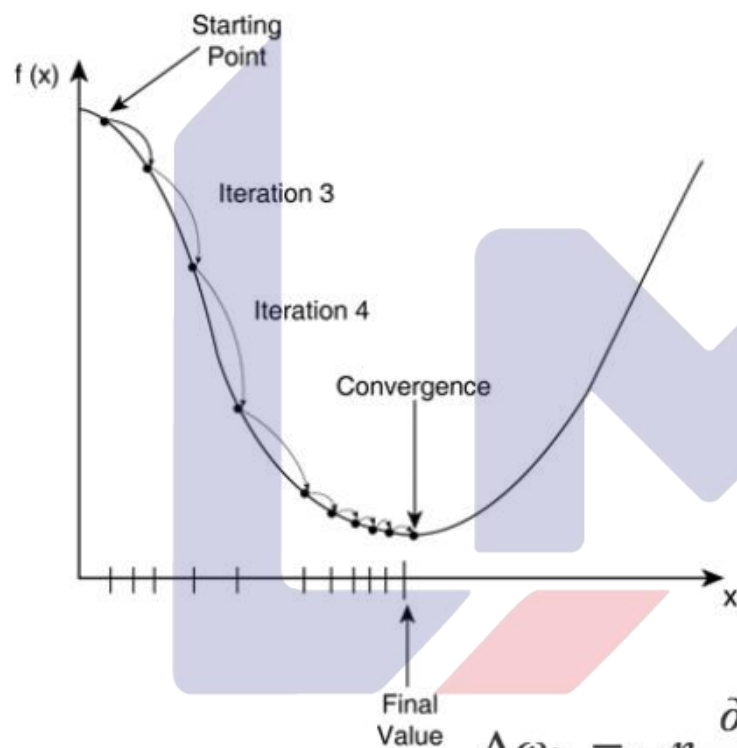
反向传播算法

- 反向传播（英语：Backpropagation，意为误差反向传播，缩写为BP）是对多层人工神经网络进行梯度下降的算法，也就是用链式法则以网络每层的权重为变量计算损失函数的梯度，以更新权重来最小化损失函数。

```
初始化网络权值（通常是小的随机值）
do
  forEach 训练样本 ex
    prediction = neural-net-output(network, ex) // 正向传递
    actual = teacher-output(ex)
    计算输出单元的误差 (prediction - actual)
    计算  $\Delta w_h$  对于所有隐藏层到输出层的权值 // 反向传递
    计算  $\Delta w_i$  对于所有输入层到隐藏层的权值 // 继续反向传递
    更新网络权值 // 输入层不会被误差估计改变
  until 所有样本正确分类或满足其他停止标准
return 该网络
```

三层网络算法（只有一个隐藏层）来源：Wiki

梯度下降

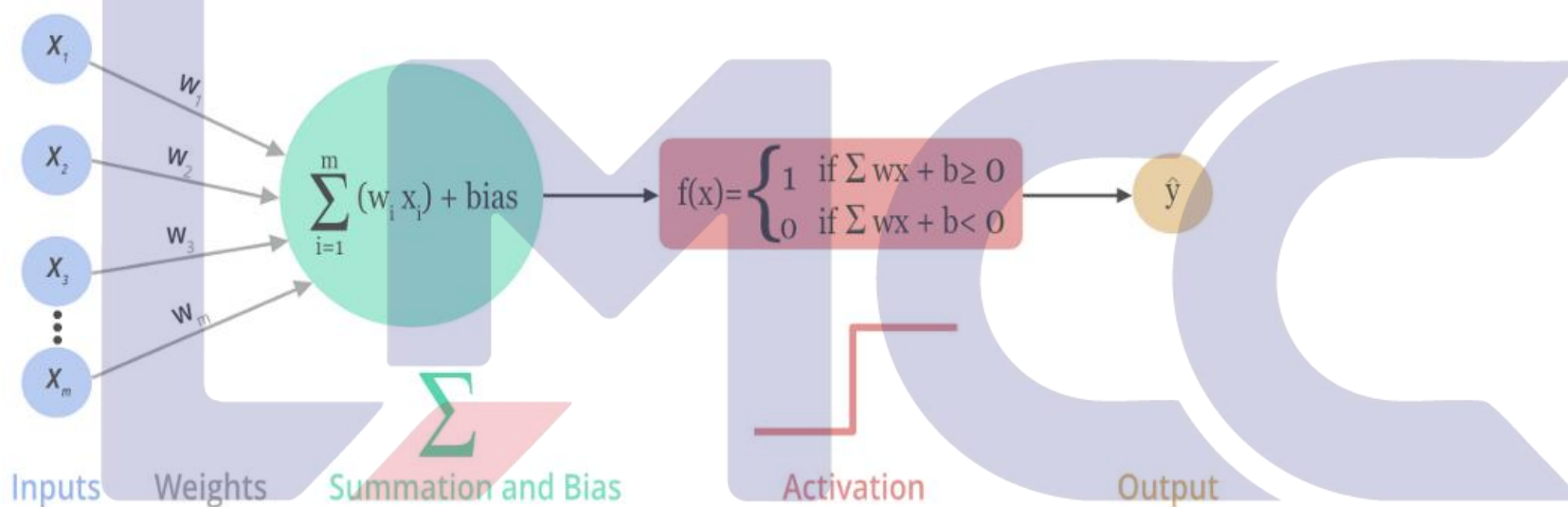


$$\Delta\omega_{j\kappa} = -\eta \frac{\partial E}{\partial \omega_{j\kappa}} \quad j = 0, 1, 2, \dots, m; \quad \kappa = 1, 2, \dots, \ell$$

$$\Delta v_{ij} = -\eta \frac{\partial E}{\partial v_{ij}} \quad i = 0, 1, 2, \dots, n; \quad j = 1, 2, \dots, m$$

激活函数

- 激活函数（Activation Function），就是在人工神经网络的神经元上运行的函数，负责将神经元的输入映射到输出端，旨在帮助网络学习数据中的复杂模式。



为什么要用激活函数

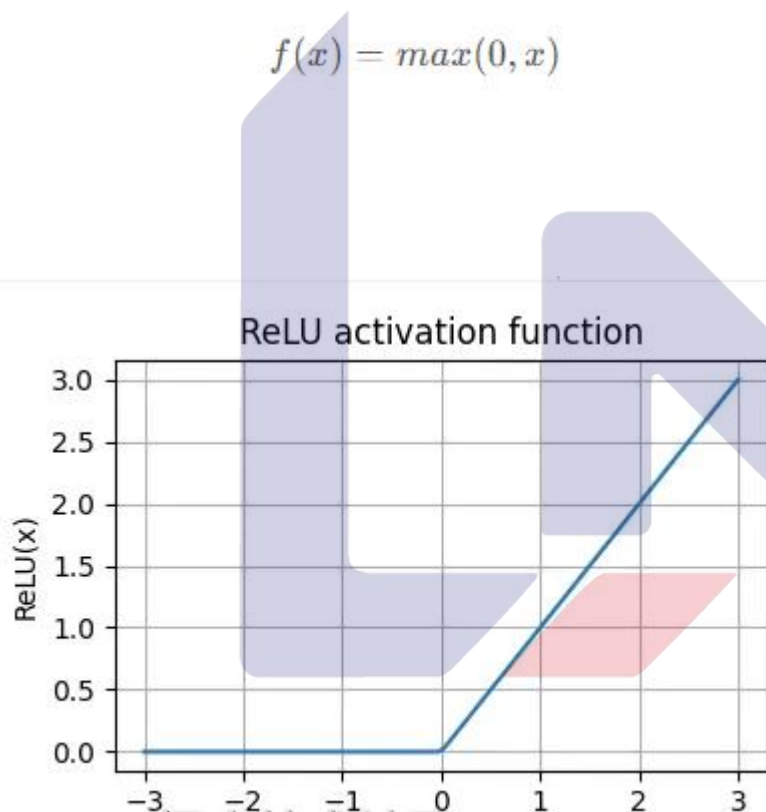
如果不用激活函数，每一层输出都是上层输入的线性函数，无论神经网络有多少层，输出都是输入的线性组合，这种情况就是最原始的感知机（Perceptron）。

使用激活函数能够给神经元引入非线性因素，使得神经网络可以任意逼近任何非线性函数，使深层神经网络表达能力更加强大，这样神经网络就可以应用到众多的非线性模型中。

不同模型的激活函数

1.ReLU函数

(Rectified Linear Unit, 线性整流单元)



优点：计算简单高效，缓解了梯度消失问题

应用： T5，原始的Transformer

2.GELU

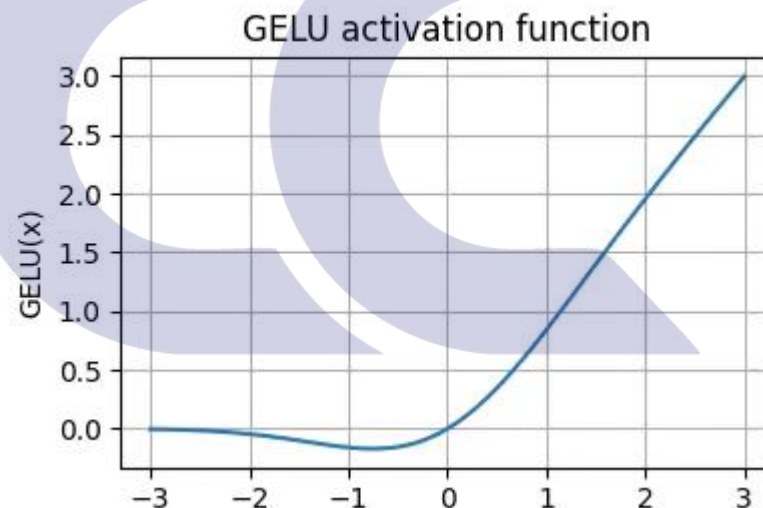
(Gaussian Error Linear Unit, 高斯误差线性单元)

GELU 的数学定义非常简洁：

$$\text{GELU}(x) = x \cdot \Phi(x)$$

其中 $\Phi(x)$ 是标准正态分布 $N(0,1)$ 的累积分布函数 (CDF)，即：

$$\Phi(x) = P(X \leq x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-t^2/2} dt$$



优点：输出连续可导，曲线更平滑，训练过程稳定

应用： ChatGPT，BERT

3.SwiGLU (Swish-Gated Linear Unit)

1.数学定义

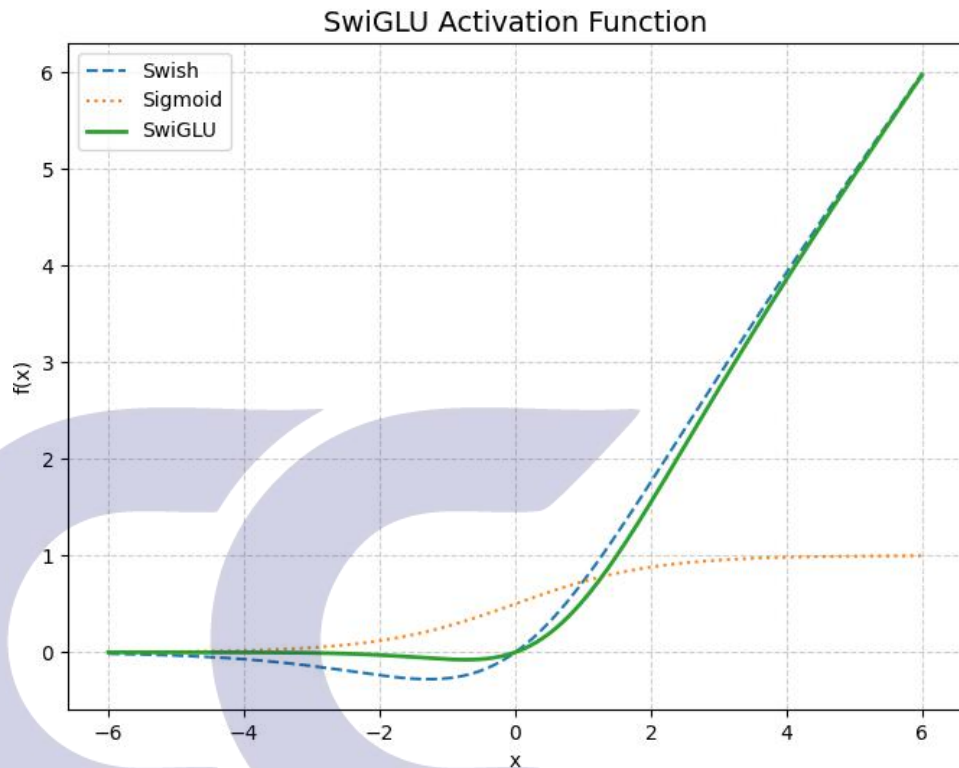
$$\text{SwiGLU}(a, b) = \text{Swish}(a) \otimes \sigma(b)$$

其中:

- $(a = xW_1 + b_1)$ 和 $(b = xW_2 + b_2)$ 是两个线性变换的输出。
- $(\text{Swish}(x) = x \cdot \text{sigmoid}(\beta x))$ ，通常取 $(\beta = 1)$ ，此时等价于 SiLU 激活函数。
- $(\otimes)$ 表示逐元素乘积 (*Hadamard*积) 。

优点：动态筛选信息，增加模型表达能力

应用：Gemin, DeepSeek



附：Sigmoid激活函数

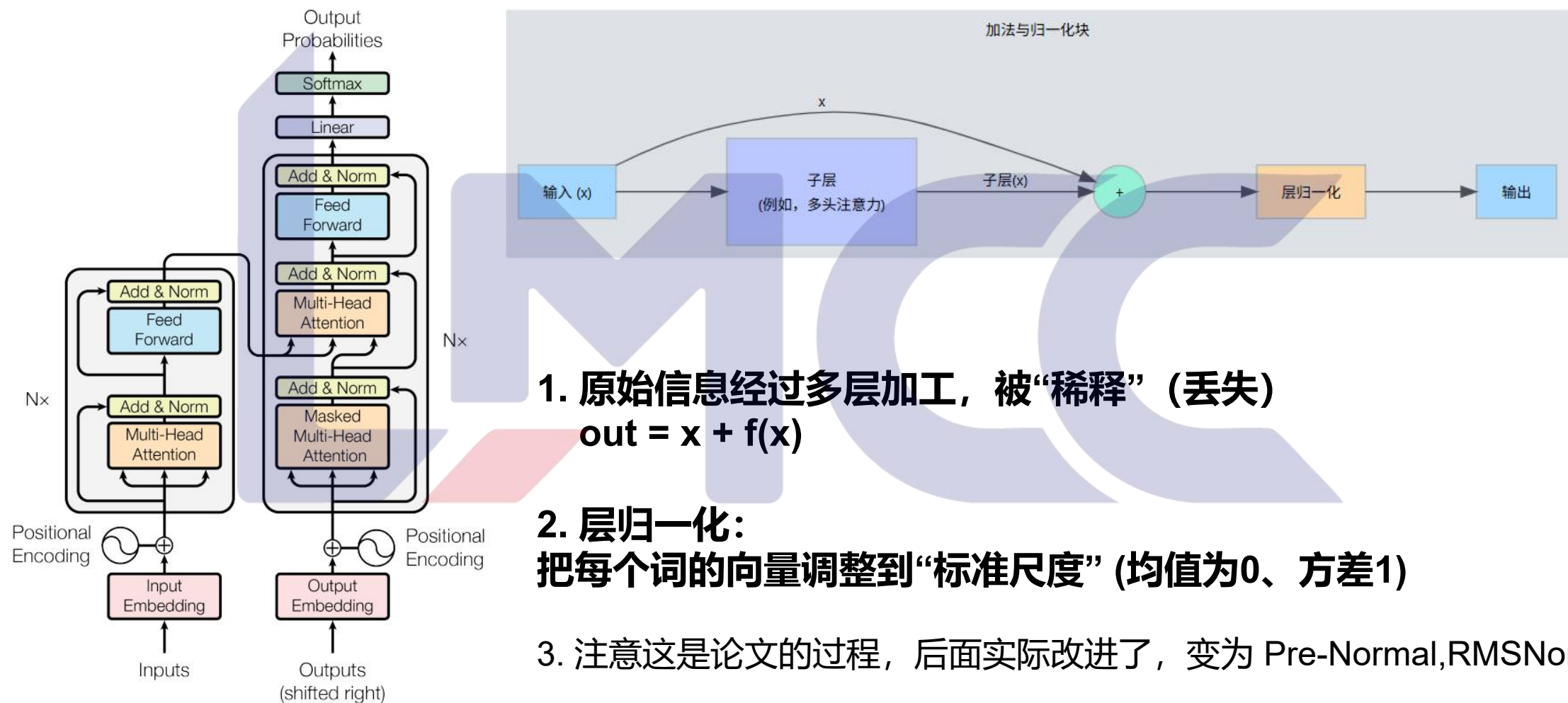
$$f(x) = \frac{1}{1 + e^{-x}}$$

想一想：无监督学习没有标签，如何学习？

- 有监督学习：有标签数据，能计算预测值与真实值的差，进而调整权重矩阵，完成学习
- 那么在没有标准答案的无监督学习中，模型又是靠什么来计算 Loss、指导权重调整的呢？



5. 相加和层归一化（残差连接和层归一化）



层归一化解决什么问题

- 深度网络训练中的一个核心困难是内部协变量偏移（Internal Covariate Shift）：每一层输入的分布在训练过程中不断变化（因为前面的层参数在更新），导致后面的层需要不断适应新的输入分布。
- 归一化通过将每层的输入重新标准化为均值为 0、方差为 1 的分布，缓解了这一问题，使每层的输入分布更加稳定，从而加速收敛并允许使用更大的学习率。

层归一化 (LayerNorm)：对每个样本，跨所有特征维度计算均值和方差，然后进行标准化。

$$\text{LN}(x_i) = \gamma \frac{x_i - \mu_L}{\sqrt{\sigma_L^2 + \epsilon}} + \beta$$

其中 μ_L 和 σ_L^2 是该样本在所有特征维度上的均值和方差。

两者都有可学习的缩放参数 γ 和偏移参数 β ，允许模型在需要时恢复原始分布。

Z-Score 标准化

$$x_i = \frac{x_i - \bar{x}}{\sigma}$$

1. 均值为 0
2. 标准差 = 1

Pre-Norm 、 RMSNorm (主流)

- 1. Transformer 原始论文使用 Post-Norm (先子层计算, 再加残差, 最后归一化) :

$$\text{output} = \text{LN}(x + \text{Sublayer}(x))$$

- 2. 后续研究发现, **Pre-Norm** (先归一化, 再子层计算, 最后加残差) 在深层模型中训练更加稳定:

$$\text{output} = x + \text{Sublayer}(\text{LN}(x))$$

- 3. **RMSNorm** (Root Mean Square Normalization), 它只保留“缩放”操作, 直接用**均方根** (RMS) 代替标准差来归一化:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} \cdot \gamma$$

主流大模型架构分类

1. 编码器-解码器架构 (Encoder-Decoder)

- ✓ 同时包含编码器和解码器，核心适配“输入-输出”类任务，比如翻译、文本摘要、问答，同时完成“理解输入”和“生成输出”两个步骤。
- ✓ 代表模型：Seq2Seq(早期)、谷歌 T5

2. 前缀解码器架构(Encoder-Only):

- ✓ 适合纯文本理解和分类任务，适合“文本理解”类任务，比如阅读理解、情感分析等
- ✓ 代表模型：谷歌 BERT、GLM

3. 因果解码器架构 (Decoder-Only) 主流架构

生成连贯、符合逻辑的文本，不需要复杂的输入理解（或仅需要简单理解）。

代表模型：GPT、LLaMA-2 等

基于 Transformer 的主流大模型架构

- 2027 年 Transformer 结构被提出后，各大科技公司和科研机构，基于 Transformer 的核心原理，研发出了多种主流大模型架构。

模型名称	开放类型	架构类别	大小	推出时间	归一化	位置编码	激活函数	L (层数)	N (头数)	H (隐藏层大小)
T5	开源	编-解	11B	2019-10	Pre RMS	Relative	ReLU	24	128	1,024
GPT-3	闭源	因果	175B	2020-05	Pre Layer	Learned	GELU	96	96	12,288
ERNIE 3.0 Titan	闭源	前缀	260B	2021-12	Pre Layer	RoPE	GeGLU	110	128	12,288
Gopher	闭源	因果	280B	2021-12	Pre Layer	Learned	GELU	80	128	16,384
UL2	开源	前缀	20B	2022-05	Pre RMS	Relative	SwiGLU	32	64	4,096
BLOOM	开源	因果	176B	2022-07	Pre Layer	ALiBi	GELU	70	112	14,336
Galactica	开源	因果	120B	2022-11	Pre Layer	Learned	GELU	96	80	10,240
ChatGLM-6B	开源	前缀	6B	2023-03	Pre RMS	RoPE	GELU	28	32	4,096
GPT-4	闭源	因果(MoE)	1.8T	2023-03	Pre Layer	RoPE	SwiGLU	120	128	8,448
MPT	开源	因果	30B	2023-05	Pre Layer	ALiBi	SwiGLU	48	64	7,168
ChatGLM2-6B	开源	前缀	6B	2023-06	Pre RMS	RoPE	SwiGLU	28	32	4,096
Mixtral 8x7B	开源	因果(MoE)	47B	2023-12	Pre RMS	RoPE	SwiGLU	32	32	4,096
InternLM-2	开源	因果	20B	2024-01	Pre RMS	RoPE	SwiGLU	48	48	6,144
GLM-4	开源	因果	9B	2024-01	Pre RMS	RoPE	SwiGLU	40	32	4,096
LLaMA-3	开源	因果	70B	2024-04	Pre RMS	RoPE	SwiGLU	80	64	8,192
Phi-3-Medium	开源	因果	14B	2024-04	Pre RMS	RoPE	SwiGLU	40	40	5,120
DeepSeek-V2	开源	因果(MoE)	236B	2024-05	Pre RMS	RoPE	SwiGLU	60	128	5,120
Gemma-2	开源	因果	27B	2024-06	Pre RMS	RoPE	GELU	46	32	4,608
LLaMA-3.1	开源	因果	405B	2024-07	Pre RMS	RoPE	SwiGLU	126	128	16,384
Qwen-2.5	开源	因果	72B	2024-09	Pre RMS	RoPE	SwiGLU	80	64	8,192
DeepSeek-V3	开源	因果(MoE)	671B	2024-12	Pre RMS	RoPE	SwiGLU	61	128	7,168



all_available_llm_ parameters_with_l

Transformer 的短板

- Transformer 的核心是自注意力机制（Self-Attention），它的效果极佳，但存在两个致命的效率短板
 - **计算复杂度为 $O(N^2)$** ：当输入的文本长度 N 增加 10 倍，自注意力的计算量会增加 100 倍。
 - **推理显存爆炸（KV Cache 问题）**：在逐字生成内容时，Transformer 需要将历史所有的 Key 和 Value 向量保存在显存中。文本越长，显存占用越大。

瓶颈：KV Cache 的显存墙

$$M_{KV} = 2 \times B \times L \times N_{layer} \times N_{head} \times d_{head} \times P_{bytes}$$

-1k tokens: 显存占用约 2.6 GB (可忽略)

-128k tokens: 显存占用约 335 GB

MQA 和 GQA 优化的重要性

B: Batch Size (批次大小)

L: 当前序列长度

N_{layer} : 层数

N_{head} : 注意力头数

d_{head} : 每头维度

P_{bytes} : 精度 (FP16 为 2 字节)

- 在自回归 (Auto-regressive) 解码过程中, 为了避免重复计算历史 Token 的 Key 和 Value, 模型会将它们缓存起来, 称为 KV Cache。

模型优化思路

- 模型存储空间复杂度是 $O(N^2)$, 中间计算复杂度 $O(N^2 d) - O(N^3)$
- 存储成本和算力成本非常高
- **优化ing...**
- 衍生很多分支

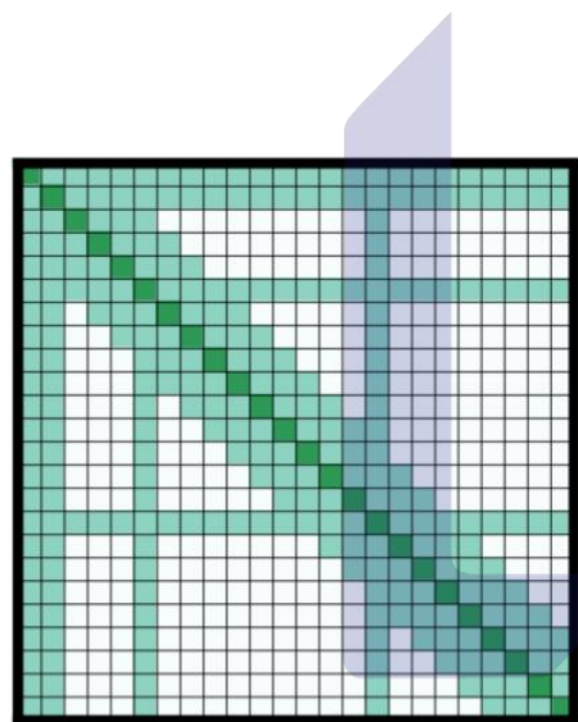


高效注意力架构：优化注意力机制

- 注意力矩阵是方阵，因此对于长度为 N 的序列，注意力机制的计算复杂度为 $O(N^2)$ 。当处理长文本时，这会成为严重的计算瓶颈。
- 研究者又提出了一些注意力机制的变体以降低计算复杂度，提高效率。



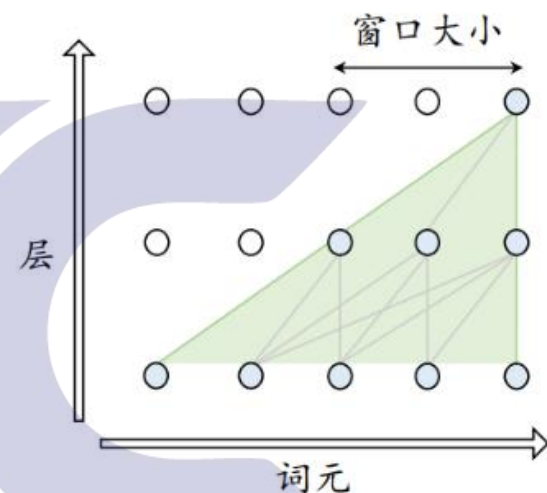
优化1. 稀疏注意力机制



注意力矩阵是对称的

中	国	人	民	大	学
1	0	0	0	0	0
1	1	0	0	0	0
1	1	1	0	0	0
0	1	1	1	0	0
0	0	1	1	1	0
0	0	0	1	1	1

(a) 滑动窗口注意力的掩码矩阵



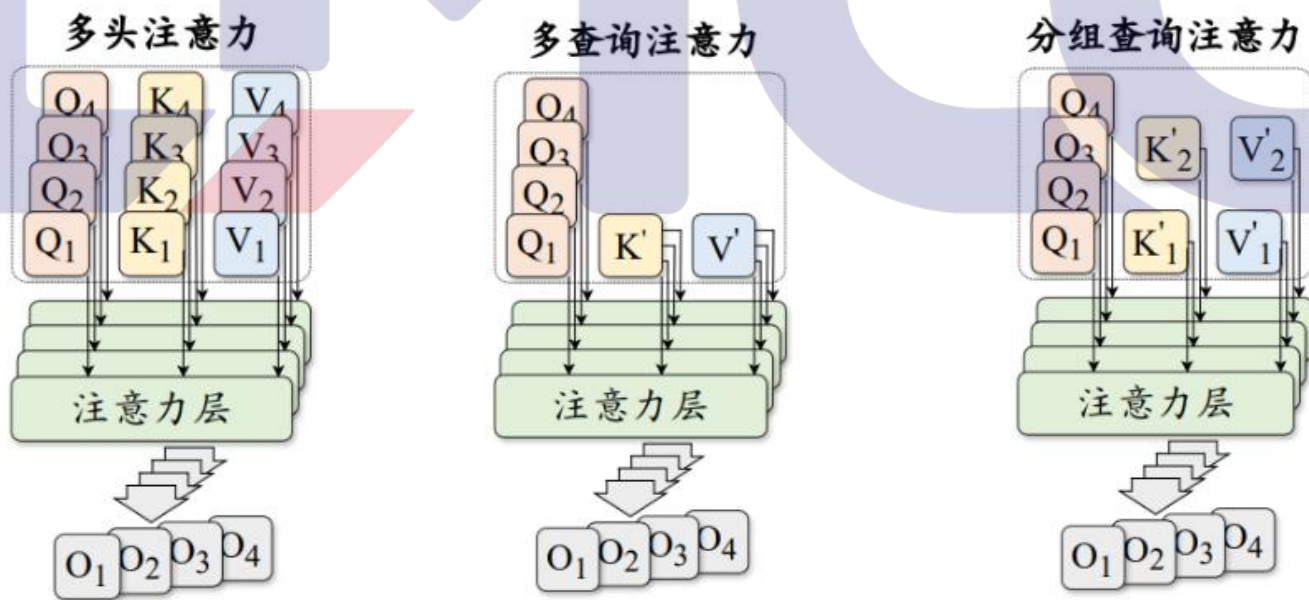
(b) 滑动窗口注意力信息的逐层传递

滑动窗口注意力机制

滑动窗口注意力机制（sliding window attention），它会设置一个长度为 w 的窗口，对于每个词元 x_t ，通过掩码机制只需要关注位于它之前窗口内的词元 $[x_{t-w+1} \dots x_t]$ ，从而将复杂度降低到 $O(nw)$ 。

优化2：多查询注意力 Multi-Query Attention, MQA

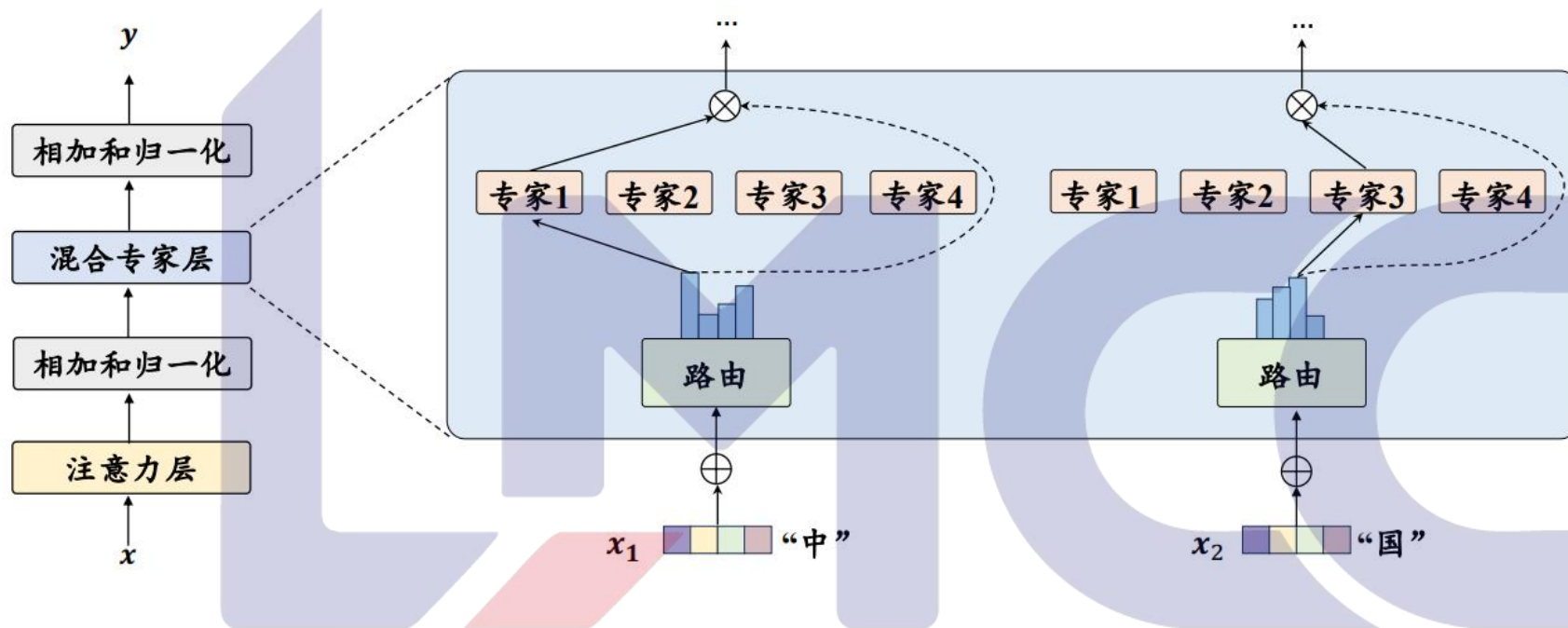
- 研究者从改进注意力机制本身出发，针对不同的头共享相同的键和值变换矩阵，这种方法通过减少访存量来实现计算速度的提升，对模型性能产生的影响也较小。这就是多查询注意力（Multi-Query Attention, MQA）
- 为了结合多查询注意力机制的效率和多头注意力机制的性能，研究者进一步提出了分组查询注意力机制（Grouped-Query Attention, GQA），分组查询注意力机制将全部的头划分为若干组，同一组内的头共享相同的变换矩阵，从而有效平衡了效率和性能。



优化3：改进计算过程优化注意力机制

- 两个具有代表性的工作是 FlashAttention 与 PagedAttention
- FlashAttention 通过矩阵分块计算以及减少内存读写次数的方式，提高了注意力分数的计算效率。
- PagedAttention 则针对解码阶段，对于键值缓存（key-value cache）进行分块存储并优化了计算方式从而提高了计算效率。

优化衍生其他模型：（1）混合专家模型



- 除了改进注意力机制以外，还有一些研究者尝试引入基于稀疏激活的混合专家架构（Mixture-of-Experts, MoE）对前馈网络模块进行改进，通过将 Transformer 模块中的特定前馈层替换为 MoE 层，使得模型可以在推理过程中仅激活部分参数，从而在不显著提升计算成本的同时实现对模型参数的拓展。

长文本优化：（2）状态空间模型(SSM)

利用隐藏层抽取长文本少量信息，即参数化状态空间，然后进行处理。

在处理极长上下文时，速度比 Transformer 更快、更省内存，

但效果比 Transformer 差，是在处理长文本信息的一种折中解决方式。

SSM 本身来自于控制工程与信号处理领域，用连续的微分方程来描述系统状态随时间的变化：

$$h'(t) = Ah(t) + Bx(t)$$

$$y(t) = Ch(t)$$

其中 $x(t)$ 是输入序列（如文本 Token）， $h(t)$ 是隐藏的状态向量（系统的“记忆”）， $y(t)$ 是输出。通过对矩阵 A, B, C 进行参数化（使其可以被神经网络学习），SSM 获得了建模复杂序列的能力。

SSM 尽管计算效率较高，但是状态空间模型的性能相比 Transformer 模型仍有差距。

尝试改进,衍生了很多其他类似模型：Mamba、RWKV、RetNet、Hyena

模型	解码复杂度	训练复杂度
Transformer	$O(TH + H^2)$	$O(T^2H + TH^2)$
Mamba	$O(N^2H + H^2)$	$O(TN^2H + TH^2)$
RWKV	$O(H^2)$	$O(TH^2)$
RetNet	$O(H^2)$	$O(TH^2)$
Hyena	$O(TM H + MH^2)$	$O(TM H \log T + TM H^2)$

T 表示序列长度， H 表示输入表示的维度， N 表示状态空间模型压缩后的维度, M 表示 Hyena 每个模块的层数

想一想：无监督学习没有标签，如何学习？

- 有监督学习：有标签数据，能计算预测值与真实值的差，进而调整权重矩阵，完成学习
- 那么在没有标准答案的无监督学习中，模型又是靠什么来计算 Loss、指导权重调整的呢？

从数据自身创造“假标签”

这是大语言模型（如 GPT、BERT）和计算机视觉中最火热的无监督范式（也常被称为自监督学习 Self-Supervised Learning）。

抠图预测 (Masked Prediction)：把输入的一部分遮住（比如掩码语言模型 Masked LM），让模型根据上下文去预测被遮住的词。这里的 Loss 就是预测词和被掩码词的交叉熵。

对比学习 (Contrastive Learning)：将同一张图片进行两次翻转/裁切（正样本对），将不同图片作为负样本。目标是拉近正样本在特征空间里的距离、推开负样本（如 InfoNCE Loss）。

参考文献

- (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30, 5998–6008.
- <https://transformers.run/>
- <https://www.wikipedia.org/>



例1 来自《CCF大模型能力认证大纲》

● 【1】 注意力机制

○ 知识点：查询 (query) - 键 (key) - 值 (value) ,
softmax 函数

题目：

关于注意力机制中查询 (Query)、键 (Key)、值 (Value) 的作用，下列说法正确的是：

- A. Query 用于存储待检索的信息，Key 用于生成最终输出
- B. 注意力分数通过 Query 与 Key 的点积计算得到，再经 softmax 归一化后对 Value 加权求和
- C. softmax 函数的作用是将注意力分数映射到 $(-\infty, +\infty)$ 的范围
- D. Value 向量决定了哪些位置应该获得更高的注意力权重

B

例2.

○ 扩展知识点：自注意力机制、交叉注意力机制

题目：

关于自注意力机制与交叉注意力机制的区别，下列描述正确的是：

- A. 自注意力机制中 Query、Key、Value 均来自同一序列，交叉注意力中 Query 来自一个序列而 Key 和 Value 来自另一个序列
- B. 自注意力机制只能用于编码器，交叉注意力只能用于解码器
- C. 交叉注意力中 Query、Key、Value 都来自同一个输入序列
- D. 自注意力机制不使用 softmax 函数进行归一化

A

例3.

●【1】主流架构

○ 知识点：编码器 - 解码器架构、因果解码器架构、前缀解码器架构

题目：

关于大语言模型的三种主流架构，下列描述正确的是：

- A. 因果解码器架构允许每个位置关注序列中所有位置的信息
- B. 编码器 - 解码器架构中，编码器使用双向注意力，解码器使用因果（单向）注意力
- C. 前缀解码器架构与因果解码器架构完全相同，没有任何区别
- D. 编码器 - 解码器架构只能处理分类任务，无法进行文本生成

B

例4.

○ 扩展知识点：架构选择与任务类型的关系（如生成任务 vs. 理解任务）

题目：

关于模型架构选择与任务类型的关系，下列说法正确的是：

- A. 编码器架构（如 BERT）更适合自然语言理解任务，因果解码器架构（如 GPT）更适合文本生成任务
- B. 因果解码器架构只能用于文本生成，完全不能做理解任务
- C. 编码器 - 解码器架构只适合机器翻译，不适合其他任何任务
- D. 所有任务都应该使用同一种架构，架构选择对性能没有影响

A

例5.

●【1】Transformer 模型的基本结构组成

○ 知识点：输入编码、位置编码、多头自注意力机制、前馈网络层、编码器、解码器

题目：

关于 Transformer 模型的基本结构组成，下列描述正确的是：

- A. Transformer 仅由注意力层组成，不包含前馈网络层
- B. Transformer 的编码器由多头自注意力层和前馈网络层交替堆叠而成，每层之后通常有残差连接和层归一化
- C. 位置编码的作用是对输入词汇进行语义编码，与位置顺序无关
- D. 多头注意力机制是指只使用一个注意力头来处理全部信息

B

例6.

○ 扩展知识点：位置编码的变体（如相对位置编码、旋转位置编码）

题目：

以下代码实现了旋转位置编码（RoPE），请阅读代码并填写空缺部分。

```
import torch
import math
```

```
def apply_rotary_pos_emb(q, k, pos):
```

```
    """
```

对 Query 和 Key 应用旋转位置编码（RoPE）

参数：

q: Query 张量, shape (batch, heads, seq_len, head_dim)

k: Key 张量, shape (batch, heads, seq_len, head_dim)

pos: 位置索引, shape (seq_len,)

返回：

旋转编码后的 q 和 k

```
    """
    head_dim = q.size(-1)
    # 计算频率基底: theta_i = 1 / (10000 ^ (2i / d))
    freq_indices = torch.arange(0, head_dim, 2, dtype=torch.float32)
    freqs = 1.0 / (10000.0 ** (freq_indices / head_dim))

    # 计算位置角度: pos * theta
    angles = pos.unsqueeze(-1).float() * freqs.unsqueeze(0) # (seq_len, head_dim//2)

    cos_vals = torch.cos(angles) # (seq_len, head_dim//2)
    sin_vals = torch.sin(angles) # (seq_len, head_dim//2)

    # 将 q 拆分为偶数维和奇数维
    q_even = q[..., 0::2] # (... , head_dim//2)
    q_odd = q[..., 1::2] # (... , head_dim//2)

    # [1] 对 Query 应用旋转变换: q_rotated_even = q_even * cos - q_odd * sin
    # [2] 对 Query 应用旋转变换: q_rotated_odd = q_even * sin + q_odd * cos
```

[1] 和 [2] 处应分别填入：

- A. [1] $q_rotated_even = q_even * cos_vals - q_odd * sin_vals$ [2] $q_rotated_odd = q_even * sin_vals + q_odd * cos_vals$
- B. [1] $q_rotated_even = q_even * sin_vals - q_odd * cos_vals$ [2] $q_rotated_odd = q_even * cos_vals + q_odd * sin_vals$
- C. [1] $q_rotated_even = q_even + cos_vals$ [2] $q_rotated_odd = q_odd + sin_vals$
- D. [1] $q_rotated_even = q_even * cos_vals + q_odd * sin_vals$ [2] $q_rotated_odd = q_even * sin_vals - q_odd * cos_vals$

A

例7.

●【2】Transformer 模型的详细配置

○ 知识点：归一化方法（LayerNorm、RMSNorm）、归一化模块位置、激活函数（GELU、Swish）、注意力机制

题目：

关于 Transformer 模型中的归一化和激活函数配置，下列说法正确的是：

- A. RMSNorm 相比 LayerNorm 省去了均值的计算，仅使用均方根进行归一化，计算效率更高
- B. LayerNorm 和 RMSNorm 的计算方式完全相同，没有区别
- C. GELU 激活函数与 ReLU 完全一致，都在负数区域输出恒为零
- D. Pre-Norm 和 Post-Norm 的区别仅在于是否使用归一化，放置位置无影响

A

例8.

●【3】混合专家模型 (MoE)

○ 知识点：稀疏激活、路由机制、负载均衡

题目：

以下代码实现了 MoE（混合专家模型）的路由机制和稀疏激活，请阅读代码并填写空缺部分。

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class MoELayer(nn.Module):

    def __init__(self, input_dim, hidden_dim, num_experts, top_k=2):
        super().__init__()
        self.num_experts = num_experts
        self.top_k = top_k
        # 路由网络：将输入映射到专家概率分布
        self.gate = nn.Linear(input_dim, num_experts, bias=False)
        # 多个专家网络
        self.experts = nn.ModuleList([
            nn.Sequential(
                nn.Linear(input_dim, hidden_dim),
                nn.ReLU(),
                nn.Linear(hidden_dim, input_dim)
            ) for _ in range(num_experts)
        ])

    def forward(self, x):
        # x: (batch_size, seq_len, input_dim)
        batch_size, seq_len, dim = x.shape
        x_flat = x.view(-1, dim) # (batch*seq, dim)

        # [1] 通过路由网络计算每个 token 对应各专家的分值，并用 softmax 归一化
        router_logits = self.gate(x_flat) # (batch*seq, num_experts)

        # [2] 选择 Top-K 个专家
```

[1] 和 [2] 处应分别填入：

- A. [1] `router_probs = F.softmax(router_logits, dim=-1)` [2] `top_k_weights, top_k_indices = torch.topk(router_probs, self.top_k, dim=-1)`
- B. [1] `router_probs = F.sigmoid(router_logits)` [2] `top_k_weights, top_k_indices = torch.topk(router_probs, self.top_k, dim=-1)`
- C. [1] `router_probs = F.softmax(router_logits, dim=0)` [2] `top_k_weights, top_k_indices = torch.sort(router_probs, dim=-1)`
- D. [1] `router_probs = F.relu(router_logits)` [2] `top_k_weights, top_k_indices = torch.topk(router_probs, self.num_experts, dim=-1)`

A

例9.

○ 扩展知识点: MoE 的显存优化与通信开销

题目:

关于 MoE 模型的显存优化与通信开销, 下列描述正确的是:

- A. MoE 模型虽然计算量与稠密模型相近, 但由于所有专家参数都需要加载, 显存占用远大于同等计算量的稠密模型
- B. MoE 模型在分布式训练中不需要跨设备通信, 因为每个专家独立工作
- C. MoE 模型的显存占用与稠密模型完全相同
- D. 专家并行 (Expert Parallelism) 会增加显存占用但不会带来通信开销

A

答案: A

解析:

- A 正确: MoE 虽然每个 token 只激活少量专家, 但所有专家参数需常驻显存
- B 错误: 专家并行需要 All-to-All 通信来路由 token 到对应的专家所在设备
- C 错误: MoE 总参数量大, 显存占用通常远大于同等计算量的稠密模型
- D 错误: 专家并行中 token 需要在设备间传输, 会带来显著的通信开销

例10.

●【3】长上下文模型

○ 知识点：位置编码拓展（如 ALiBi）、长上下文窗口拓展、长上下文数据构建

关于长上下文模型中的位置编码拓展方法,下列说法正确的是:

- A. ALiBi (Attention with Linear Biases) 不使用位置编码向量,而是在注意力分数上直接加上与距离成正比的线性偏置
- B. 所有位置编码方法都无法处理超出训练时最大长度的序列
- C. 长上下文窗口拓展只需要修改位置编码,不需要使用长文本数据进行额外训练
- D. ALiBi 需要为每个位置学习独立的位置嵌入向量

答案: A

解析:

- A 正确: ALiBi 在注意力分数上减去一个与 query-key 距离成正比的偏置项,距离越远惩罚越大
- B 错误: ALiBi、RoPE 插值等方法可以较好地外推到更长序列
- C 错误: 长上下文拓展通常还需要长文本数据进行继续训练以适应长序列
- D 错误: ALiBi 是预定义的线性偏置,无需学习位置嵌入

例 11.

○ 扩展知识点: 长上下文模型的显存与计算效率优化、
长上下文模型的训练方法

题目:

关于长上下文模型的效率优化和训练方法, 下列描述正确

的是:

- A. 标准自注意力的计算复杂度与序列长度呈线性关系
- B. 长上下文模型通常采用渐进式训练策略, 先在短序列上训练再逐步增加序列长度
- C. 增加上下文窗口长度不会增加任何额外的显存消耗
- D. 长上下文模型只需要使用短文本数据就可以获得良好的长文本处理能力

答案: B

解析:

- A 错误: 标准自注意力的计算复杂度与序列长度呈二次方关系 $O(n^2)$
- B 正确: 渐进式训练 (逐步增加序列长度) 是常用的长上下文训练策略
- C 错误: 更长的上下文会显著增加 KV 缓存的显存消耗
- D 错误: 长上下文模型需要高质量的长文本数据进行训练

例 12.

● [3] 高效注意力架构

○ 知识点：稀疏注意力、线性注意力、键值缓存优化、多查询注意力（MQA）、分组查询注意力（GQA）、多头潜在注意力（MLA）

题目：

以下代码实现了分组查询注意力（GQA），请阅读代码并填写空缺部分。

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class GroupedQueryAttention(nn.Module):
    """ 分组查询注意力（GQA）：多个 Query 头共享同一组 Key 和 Value 头 """
    def __init__(self, hidden_dim, num_q_heads, num_kv_heads):
        super().__init__()
        self.num_q_heads = num_q_heads # Query 头的数量，例如 32
        self.num_kv_heads = num_kv_heads # KV 头的数量，例如 8
        self.head_dim = hidden_dim // num_q_heads
        # [1] 计算每个 KV 头对应多少个 Query 头（分组大小）

        self.q_proj = nn.Linear(hidden_dim, num_q_heads * self.head_dim)
        self.k_proj = nn.Linear(hidden_dim, num_kv_heads * self.head_dim)
        self.v_proj = nn.Linear(hidden_dim, num_kv_heads * self.head_dim)
        self.o_proj = nn.Linear(hidden_dim, hidden_dim)

    def forward(self, x):
        B, L, _ = x.shape

        q = self.q_proj(x).view(B, L, self.num_q_heads, self.head_dim).transpose(1, 2)
        k = self.k_proj(x).view(B, L, self.num_kv_heads, self.head_dim).transpose(1, 2)
        v = self.v_proj(x).view(B, L, self.num_kv_heads, self.head_dim).transpose(1, 2)
```

[1] 和 [2] 处应分别填入：

- A. [1] `self.num_groups = num_q_heads // num_kv_heads` [2] `k = k.repeat_interleave(self.num_groups, dim=1)` 和 `v = v.repeat_interleave(self.num_groups, dim=1)`
- B. [1] `self.num_groups = num_kv_heads // num_q_heads` [2] `k = k.repeat(1, self.num_groups, 1, 1)` 和 `v = v.repeat(1, self.num_groups, 1, 1)`
- C. [1] `self.num_groups = num_q_heads // num_kv_heads` [2] `k = k.expand(B, self.num_q_heads, L, self.head_dim)` 和 `v = v.expand(B, self.num_q_heads, L, self.head_dim)`
- D. [1] `self.num_groups = num_q_heads * num_kv_heads` [2] `k = k.repeat_interleave(self.num_groups, dim=1)` 和 `v = v.repeat_interleave(self.num_groups, dim=1)`

A

题干有误

例13. 来源于 《LMCC-青少年组-新例卷》

1. 关于监督学习、无监督学习与自监督学习，下列说法正确的是：

- A. 监督学习主要依赖无标签数据，自监督学习通常需要人工标注
- B. 聚类通常属于监督学习，分类通常属于无监督学习
- C. 自监督学习通常从数据本身构造训练信号，例如语言模型的下一个词元预测
- D. 模型参数量增大后，训练数据的重要性会明显降低

例14.

4. 关于注意力机制中查询 (Query)、键 (Key)、值 (Value) 的作用，下列说法正确的是：

- A. Query 用于存储待检索的信息，Key 用于生成最终输出
- B. 注意力分数通过 Query 与 Key 的点积计算得到，再经 softmax 归一化后对 Value 加权求和
- C. softmax 函数的作用是将注意力分数映射到 $(-\infty, +\infty)$ 的范围
- D. Value 向量决定了哪些位置应该获得更高的注意力权重

B

例15.

6. 关于大语言模型的“生成”和“理解”两类核心范式，下列说法正确的是：

- A. 生成任务更侧重让模型产生新的文本，理解任务更侧重分析或判断输入内容的含义
- B. 生成任务主要用于判断文本类别，理解任务主要用于续写故事或扩写文章
- C. 生成和理解的差异主要体现在输出文字长短，与任务目标关系较弱
- D. 大语言模型更适合生成任务，用于理解任务时参考价值较低

A

例16.

11. 关于语言模型的发展历程，下列理解最准确的是：

- A. 统计语言模型、神经网络语言模型、预训练语言模型和大语言模型大致体现了语言模型能力逐步增强的发展路线
- B. 大语言模型先出现，随后才发展出神经网络语言模型和统计语言模型
- C. 预训练语言模型主要依赖人工规则库，对从语料中学习语言规律的依赖较低
- D. 统计语言模型和大语言模型的主要区别在于是否用于中文任务

A